



Figure 1 A changer une fois l'horloge fonctionnel par l'image de cette dernière

TPI : HORLOGE À LED AVEC CAPTEUR

Documentation du projet

Chef de projet

Raphael FAVRE

Enseignants aux CPNV en informatique

Experts

Süleyman CERAN

Ingénieur Systèmes et Réseaux, Administrateur Systèmes à UNIL

Alexandre GRAF

Travaille à Omicron Web Services SA

Ryan Bersier

SI-C4a – CPNV - 2026

Table des matières

1	ANALYSE PRÉLIMINAIRE	2
1.1	INTRODUCTION.....	2
1.2	OBJECTIFS.....	2
1.3	ANALYSE DES BESOINS MATÉRIELS ET LOGICIELS.....	5
1.4	ANALYSE DES RISQUES.....	11
1.5	PLANIFICATION INITIALE.....	15
2	ANALYSE	22
2.1	JUSTIFICATION CHOIX DE L'IDE	22
2.2	ANALYSE DES ÉLÉMENTS ÉLECTRONIQUE	23
2.3	CONTRAINTES TECHNIQUES	26
3	CONCEPTION	29
3.1	SCHÉMA ÉLECTRONIQUE	29
3.2	DÉFINITION DE L'ARCHITECTURE LOGICIEL	30
3.3	CONCEPTION DES MODULES	32
3.4	DIAGRAMMES DE FLUX	34
3.5	STRATÉGIE DE TEST	36
3.6	PLANIFICATION FINALE	39
3.7	DOSSIER DE CONCEPTION.....	39
4	RÉALISATION.....	50
4.1	DOSSIER DE RÉALISATION	50
4.2	DESCRIPTION DES TESTS EFFECTUÉS.....	ERREUR ! SIGNET NON DEFINI.
4.3	ERREURS RESTANTES.....	51
4.4	LISTE DES DOCUMENTS FOURNIS.....	51
5	CONCLUSIONS	51
6	ANNEXES.....	51
6.1	RÉSUMÉ DU RAPPORT DU TPI / VERSION SUCCINCTE DE LA DOCUMENTATION	52
6.2	SOURCES – BIBLIOGRAPHIE	52
6.3	JOURNAL DE TRAVAIL	52
6.4	MANUEL D'INSTALLATION.....	52
6.5	MANUEL D'UTILISATION.....	52
6.6	ARCHIVES DU PROJET	52

1 Analyse préliminaire

1.1 Introduction

Ce projet s'inscrit dans le cadre du Travail de Projet Individuel (TPI) de la formation d'Informaticien CFC (Ordonnance 2021), orientation Exploitation et infrastructure, réalisé au CPNV à Sainte-Croix. Il porte sur la conception et la réalisation d'une horloge murale à LED avec capteurs environnementaux, pilotée par une carte Arduino.

L'idée de départ part d'un constat concret : de nombreuses salles de classe du CPNV sont équipées de petits boîtiers mesurant le taux de CO₂ dans l'air, afin de surveiller la qualité de l'environnement. Ces appareils effectuent leur travail, mais ils souffrent d'un défaut important : ils n'émettent aucune alerte visuelle ou sonore lorsque le taux devient problématique. Résultat, les personnes présentes dans la salle doivent penser à regarder l'appareil régulièrement pour savoir si l'air est encore acceptable ce qui, en pratique, n'arrive pas toujours.

Ce projet vise à pallier ce manque en proposant un appareil qui combine deux fonctions en une : d'un côté, une horloge murale affichant l'heure en temps réel grâce à un anneau de LED NeoPixel animé, et de l'autre, un système de surveillance environnementale affichant en continu la température, l'humidité, la pression atmosphérique et la qualité de l'air (COV) sur un écran alphanumérique. Dès que la qualité de l'air se dégrade, l'anneau LED change de comportement pour émettre une alerte visuelle immédiate et impossible à ignorer.

L'appareil repose sur plusieurs composants matériels mis à disposition : une carte Arduino REV3, quatre anneaux Adafruit NeoPixel formant un cercle de 60 LED, un capteur environnemental BME680, un module d'horloge temps réel RTC DS1307 garantissant la précision de l'heure même en cas de coupure de courant, un affichage Adafruit 4×14 segments, ainsi que six boutons poussoirs permettant à l'utilisateur de régler l'heure, d'ajuster la luminosité et de changer le mode d'affichage.

Ce projet représente une réelle opportunité d'apprentissage dans le domaine des systèmes embarqués, en combinant des aspects de programmation bas niveau, de câblage électronique, de gestion de capteurs et de conception d'une interface utilisateur simple mais efficace. Il s'appuie sur des compétences acquises notamment dans les modules INI-03, MA-20, 319, ainsi que sur le projet Système Embarqué en Arduino déjà réalisé en formation.

Aucun travail préliminaire spécifique à ce projet n'avait été effectué avant le début du TPI. L'intégralité de la conception, du câblage et du développement est réalisée durant la période du 23 avril au 21 mai 2026, dans les locaux du CPNV.

1.2 Objectifs

1.2.1 Objectifs du projet

Les objectifs suivants ont été définis sur la base du cahier des charges. Chacun d'eux devra pouvoir être vérifié et validé à la fin du projet, que ce soit par démonstration directe, par les résultats des tests ou par la lecture du rapport.

1.2.1.1 Objectif 1 Affichage de l'heure en temps réel

L'horloge doit afficher l'heure courante (heures, minutes, secondes) de manière fluide et lisible sur l'anneau de 60 LED NeoPixel, composé de quatre quarts d'anneau assemblés. L'affichage doit rester correct même après une coupure de courant, grâce au module RTC DS1307.

1.2.1.2 Objectif 2 Mesure et affichage des données environnementales

Le système doit lire en continu les données du capteur BME680 température, humidité, pression atmosphérique et qualité de l'air (COV) et les afficher de manière alternée et claire sur l'écran alphanumérique 4×14 segments.

1.2.1.3 Objectif 3 Alerte visuelle en cas de mauvaise qualité de l'air

Lorsque le niveau de COV dépasse un seuil défini, l'anneau de LED doit réagir visuellement de façon distincte (changement de couleur, clignotement) afin d'alerter immédiatement les personnes présentes dans la salle, sans qu'elles aient besoin de regarder l'écran.

1.2.1.4 Objectif 4 Interaction utilisateur via les boutons poussoirs

Les six boutons poussoirs doivent permettre à l'utilisateur de régler manuellement l'heure, de changer le mode d'affichage de l'écran alphanumérique et d'ajuster la luminosité des LED, avec une gestion correcte du rebond (debounce).

1.2.1.5 Objectif 5 Qualité et maintenabilité du code

Le code Arduino doit être structuré en modules distincts (gestion RTC, NeoPixel, BME680, HT16K33, boutons), commenté et versionné sur un dépôt GitHub mis à jour quotidiennement.

1.2.1.6 Objectif 6 Documentation complète

Un rapport de projet complet, un journal de travail, une procédure d'installation et de mise en service, ainsi qu'une archive du projet doit être livrés dans les délais définis par le cahier des charges.

1.2.2 Objectif personnel

1.2.2.1 Me dépasser sur un projet complet

C'est la première fois que je gère un projet de cette ampleur seul, du début à la fin. Je veux prouver surtout à moi-même que j'en suis capable.

1.2.2.2 Progresser autant sur le plan technique qu'humain

Gérer mon stress, respecter mes délais, prendre des décisions seul quand ça coince ce sont des compétences que je veux autant développer que la programmation ou le câblage.

1.2.2.3 Apprendre à résoudre des problèmes inconnus

Ce projet va forcément me mettre face à des situations nouvelles. Mon objectif est d'apprendre à chercher, tester et trouver par moi-même.

1.2.2.4 Mieux gérer mon temps et ma planification

J'ai tendance à sous-estimer certaines tâches. Ce TPI est une occasion concrète d'apprendre à planifier de façon réaliste et à ajuster quand les choses ne se passent pas comme prévu.

1.2.2.5 Comprendre et maîtriser le protocole I²C

Je veux vraiment comprendre comment plusieurs composants cohabitent sur le même bus, comment détecter les conflits d'adresses et comment diagnostiquer un problème de communication pas juste utiliser des bibliothèques sans savoir ce qu'il se passe dessous.

1.2.2.6 Écrire un code structuré et modulaire

Je veux organiser mon code en modules séparés et indépendants, avec des fonctions claires et des commentaires utiles comme le ferait un développeur professionnel sur un vrai projet embarqué.

1.2.2.7 Maîtriser la brasure et le câblage électronique

C'est la partie qui m'inquiète le plus et donc celle où je veux le plus progresser. Je veux sortir de ce projet à l'aise avec le fer à souder et capable de monter un circuit proprement.

1.2.2.8 Créer quelque chose de concret et utile

Ce qui me motive profondément c'est de produire un objet réel qui fonctionne et qui a du sens pas juste un exercice scolaire qu'on oublie le lendemain.

1.2.2.9 Apprendre à utiliser Git de façon rigoureuse

Commiter régulièrement, écrire des messages de commit clairs, garder un historique propre je veux prendre de vraies bonnes habitudes de versioning que je garderai toute ma carrière.

1.2.2.10 Apprendre à documenter mon travail au fil de l'eau

Plutôt que de tout rédiger en catastrophe à la fin, je veux prendre l'habitude d'écrire au fur et à mesure une discipline que je veux garder bien après ce TPI.

1.2.2.11 Comprendre la gestion du temps réel sans système d'exploitation

Gérer plusieurs tâches simultanées sur Arduino sans bloquer l'exécution avec des `delay()` est un vrai défi. Je veux maîtriser la logique non-bloquante avec `millis()` et comprendre comment prioriser les tâches sur un seul cœur.

1.2.2.12 Gagner en autonomie et en confiance

Chaque problème résolu seul, chaque décision technique assumée je veux que ce projet me rende plus confiant dans mes capacités pour la suite de ma carrière.

1.2.2.13 Soigner les détails jusqu'au bout

Pas seulement que ça fonctionne, mais que le câblage soit propre, le code lisible et le rapport honnête. Je veux pouvoir regarder le résultat final et me dire que j'ai vraiment donné le meilleur de moi-même.

1.2.2.14 Profiter de cette expérience unique

Un TPI ne se fait qu'une fois. Au-delà des notes et des évaluations, je veux vivre ce projet comme une vraie expérience professionnelle dont je me souviendrai.

1.3 Analyse des besoins matériels et logiciels

Avant de commencer toute phase de conception ou de réalisation, il est essentiel d'identifier et de comprendre l'ensemble des ressources nécessaires à la réalisation du projet. Cette section passe en revue le matériel mis à disposition ainsi que les outils logiciels qui seront utilisés tout au long du développement.

1.3.1 Besoins matériels

1.3.1.1 Carte Arduino REV3

La carte Arduino REV3 constitue le cœur du projet. C'est elle qui orchestre l'ensemble du système : elle lit les données des capteurs, calcule l'affichage de l'heure, pilote les LED et réagit aux actions de l'utilisateur sur les boutons. Elle communique avec les différents composants via les protocoles I²C et les broches numériques/analogiques disponibles.



Figure 2 Photo de l'Arduino et de sa Breadboard prise à la main

1.3.1.2 Anneaux Adafruit NeoPixel 1/4 — 60 LED au total x4

Ces quatre quarts d'anneau, assemblés pour former un cercle complet de 60 LED RGB, constituent l'affichage principal de l'horloge. Chaque LED est adressable individuellement, ce qui permet de créer des animations fluides pour représenter les heures, les minutes et les secondes, mais aussi de changer l'apparence de l'anneau

en cas d'alerte environnementale. Leur pilotage se fait via une seule broche de données et la bibliothèque Adafruit NeoPixel.



Figure 3 Photo des 4 quart de l'anneau de led avec le dernier quart retourner pour bien pouvoir voir les connection

1.3.1.3 Capteur environnemental BME680

Ce capteur multifonction est capable de mesurer simultanément la température ambiante, l'humidité relative, la pression atmosphérique et la qualité de l'air via un indice de composés organiques volatils (COV). Il communique avec l'Arduino via le bus I²C et constitue la source de toutes les données environnementales affichées et surveillées par le système.



Figure 4 Photo du capteur environnemental

1.3.1.4 Module horloge temps réel RTC DS1307

Le module RTC DS1307 est une horloge temps réel qui maintient l'heure exacte de manière autonome, même en cas de coupure de courant, grâce à une pile de sauvegarde intégrée. Il communique également via I²C et garantit que l'horloge affichée reste toujours précise, sans dérive, tout au long de l'utilisation de l'appareil.

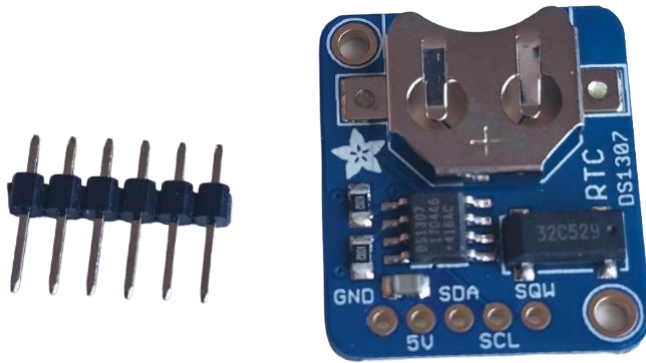


Figure 5 Photo du Module horloge pour garder la date et l'heure en mémoire

1.3.1.5 Affichage Adafruit 4×14 segments alphanumérique HT16K33

Cet affichage alphanumérique, basé sur le contrôleur HT16K33 et pilotable en I²C, permet d'afficher du texte et des chiffres. Il est utilisé pour présenter en alternance les différentes mesures environnementales (température, humidité, pression, qualité de l'air) de façon lisible et compacte.

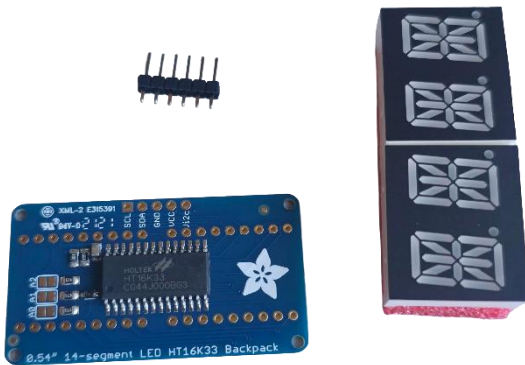


Figure 6 Photo de l'affichage 14 segment pas encore assembler

1.3.1.6 Boutons poussoirs x6

Les six boutons permettent à l'utilisateur d'interagir directement avec le système. Ils servent à régler manuellement l'heure, à changer le mode d'affichage de l'écran alphanumérique, et à ajuster la luminosité des LED. Une gestion du rebond (debounce) devra être implémentée pour garantir un comportement fiable à chaque pression.



Figure 7 Photo des 6 boutons pour le projet

1.3.1.7 Poste de brasure et composants électroniques associés

Le projet nécessite un travail de câblage et de brasure pour assembler les différents composants de manière solide et durable. Des résistances, condensateurs, fils de connexion et une breadboard sont mis à disposition pour réaliser le montage électronique.

1.3.1.8 Cable USB-B 2.0 vers USB-A 2.0

Ce câble est indispensable pour connecter la carte Arduino à l'ordinateur. Il sert à la fois à alimenter la carte durant le développement et à téléverser le programme compilé depuis CLion/PlatformIO directement sur le microcontrôleur. Sans ce câble, aucun transfert de code n'est possible entre l'environnement de développement et la carte.



Figure 8 Photo câble fourni usb-a vers usb-b

1.3.1.9 Cable de connexion pin à pin mâle

Ces fils de connexion, aussi appelés jumpers, sont utilisés pour relier les différents composants électroniques entre eux sur la breadboard et vers les broches de l'Arduino. Ils permettent de créer le câblage de manière propre, flexible et modifiable sans brasure, ce qui est particulièrement utile durant la phase de prototypage et de tests.

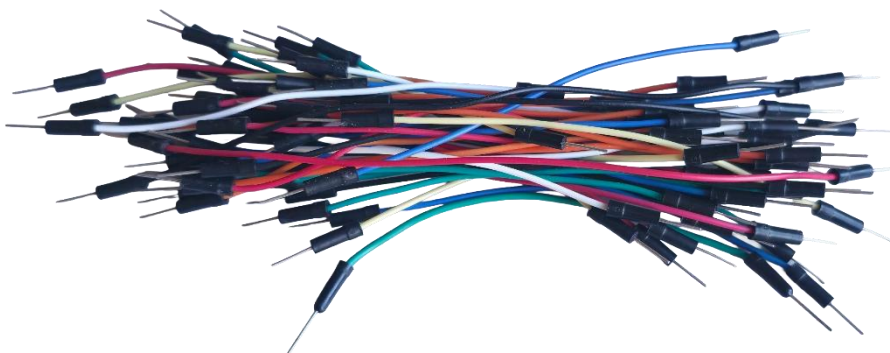


Figure 9 photo de cable de connexion pin à pin mâle en pagaille

1.3.1.10 Ordinateur du CPNV

L'ordinateur mis à disposition par le CPNV constitue la station de travail principale pour l'ensemble du projet. Il héberge l'environnement de développement CLion avec PlatformIO, les outils de versioning Git, la suite Microsoft Office pour la rédaction des documents, ainsi que tous les fichiers du projet. C'est depuis cet ordinateur que le

code est écrit, compilé et téléversé sur la carte Arduino tout au long de la période de réalisation.



Figure 10 PC fourni par le CPNV OptiPlex 7000 DELL

1.3.2 Besoins logiciels

1.3.2.1 CLion avec l'extension PlatformIO

Pour le développement du code embarqué, j'ai choisi d'utiliser CLion, l'IDE de JetBrains, couplé à l'extension PlatformIO. Ce choix s'écarte de l'IDE Arduino classique, mais offre des avantages significatifs pour un projet de cette envergure : un éditeur de code puissant avec autocomplétion intelligente, une gestion propre des bibliothèques via PlatformIO, un débogage facilité, et une meilleure organisation du projet en fichiers et modules distincts. PlatformIO prend en charge nativement les cartes Arduino et gère automatiquement les dépendances, ce qui simplifie l'intégration des bibliothèques tierces.

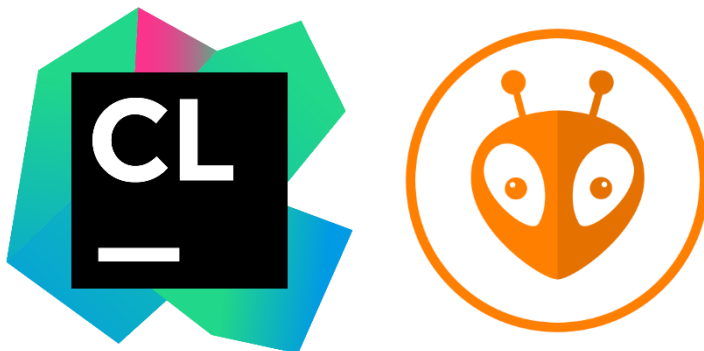


Figure 11 Image du logo de CLion et du logo de son plugin PlatformIO

1.3.2.2 Bibliothèque Adafruit NeoPixel

Cette bibliothèque officielle d'Adafruit est indispensable pour piloter les anneaux LED. Elle permet de contrôler chaque LED individuellement, de définir des couleurs en RGB et de créer des animations. Elle sera intégrée au projet via le gestionnaire de bibliothèques de PlatformIO.

1.3.2.3 Bibliothèque Adafruit BME680

La bibliothèque Adafruit pour le capteur BME680 fournit une interface simple et fiable pour lire toutes les données du capteur (température, humidité, pression, COV) sans avoir à gérer manuellement les registres I²C de bas niveau.

1.3.2.4 Bibliothèque RTCLib

La bibliothèque RTCLib, développée par Adafruit, facilite la communication avec le module RTC DS1307. Elle permet de lire et d'écrire l'heure courante de façon claire, et d'effectuer des conversions de dates et d'heures sans complexité inutile.

1.3.2.5 Bibliothèque Adafruit LEDBackpack / GFX

Ces bibliothèques permettent de piloter l'affichage alphanumérique HT16K33 via I²C. Elles offrent des fonctions simples pour écrire du texte ou des valeurs numériques sur les segments, ce qui rend l'affichage des données environnementales straightforward à implémenter.

1.3.2.6 Git et GitHub

L'ensemble du code source sera versionné avec Git et publié sur un dépôt GitHub mis à jour quotidiennement, conformément aux exigences du cahier des charges. Cela garantit un suivi précis de l'évolution du projet, permet de revenir à une version antérieure en cas de problème, et constitue une archive complète du travail effectué.



Figure 12 Image du logo de git et de github

1.3.2.7 Suite Microsoft Office

Microsoft Word sera utilisé pour la rédaction du rapport de projet et du journal de travail, conformément aux livrables attendus. Microsoft PowerPoint pourra être utilisé pour préparer la présentation orale prévue les 2 ou 3 juin 2026.



Figure 13 Image du logo de Microsoft Office

1.4 Analyse des risques

Avant de se lancer dans la phase de conception et de réalisation, il est important d'identifier les risques potentiels qui pourraient compromettre le bon déroulement du projet. Cette analyse permet d'anticiper les problèmes, de préparer des solutions de repli et d'aborder chaque étape avec plus de sérénité.

Voici la section complète sur l'analyse des risques, en intégrant tes inquiétudes sur la brasure et les branchements.

1.4.1 Risque 1

Erreur de câblage ou de brasure

Niveau de risque : Élevé

C'est le risque qui m'inquiète le plus dans ce projet. Le montage implique de nombreux composants reliés entre eux via des câbles, des broches et des points de brasure. Une erreur de câblage comme inverser l'alimentation et la masse (GND/VCC) ou une mauvaise brasure comme un faux contact ou un court-circuit peut endommager irrémédiablement un composant, voire la carte Arduino elle-même.

Mesures préventives :

Avant chaque branchement, je vérifierai systématiquement le schéma de câblage. Je procéderai composant par composant, en testant le bon fonctionnement de chacun avant de passer au suivant. Pour la brasure, je m'assurerai que les joints sont propres, brillants et bien formés, et j'utiliserai un multimètre pour vérifier la continuité électrique et l'absence de court-circuit avant toute mise sous tension.

1.4.2 Risque 2

Composant défectueux ou endommagé

Niveau de risque : Moyen

Malgré toutes les précautions prises, un composant peut s'avérer défectueux dès le départ ou être endommagé lors du montage (surchauffe à la brasure, électricité

statique, mauvaise manipulation). Si un module clé comme le BME680 ou le RTC ne fonctionne pas, cela peut bloquer une partie importante du développement.

Mesures préventives :

Je testerai chaque composant individuellement dès son branchement, à l'aide de programmes de test simples, avant de les intégrer dans le projet global. En cas de panne avérée, j'en informerai immédiatement mon chef de projet afin d'obtenir un composant de remplacement dans les meilleurs délais.

1.4.3 Risque 3

Conflit ou incompatibilité entre les bibliothèques

Niveau de risque : Moyen

Le projet utilise plusieurs bibliothèques tierces simultanément : Adafruit NeoPixel, Adafruit BME680, RTCLib et Adafruit LEDBackpack. Il est possible que certaines versions de ces bibliothèques entrent en conflit, consomment trop de mémoire RAM ou Flash sur l'Arduino, ou provoquent des comportements inattendus lorsqu'elles sont utilisées ensemble.

Mesures préventives :

J'intégrerai les bibliothèques progressivement, en testant la compatibilité à chaque ajout. Je consulterai la documentation officielle d'Adafruit et les issues GitHub des bibliothèques concernées pour m'assurer d'utiliser des versions stables et compatibles. En cas de problème de mémoire, j'optimiserai le code en réduisant l'utilisation des variables globales et en privilégiant les types de données les plus légers.

1.4.4 Risque 4

Dépassement du budget temps

Niveau de risque : Moyen

Avec 90 heures disponibles et un projet qui touche à la fois au matériel et au logiciel, le risque de prendre du retard est réel. Certaines tâches peuvent s'avérer plus longues que prévu notamment le débogage, l'intégration des modules ou la rédaction du rapport et empiéter sur les étapes suivantes.

Mesures préventives :

Je suivrai rigoureusement ma planification initiale et mettrai à jour mon journal de travail quotidiennement pour détecter tout écart rapidement. Si une tâche prend trop de temps, je l'évaluerai et déciderai si je peux la simplifier ou la reporter, en priorisant toujours les fonctionnalités essentielles (affichage de l'heure, lecture des capteurs) avant les fonctionnalités secondaires (animations, modes d'affichage avancés).

1.4.5 Risque 5

Problème de communication I²C entre les composants

Niveau de risque : Moyen

Trois composants du projet (BME680, RTC DS1307 et HT16K33) communiquent tous via le bus I²C. Si deux composants partagent la même adresse I²C ou si le bus est mal configuré, les communications peuvent échouer ou se mélanger, rendant les données illisibles.

Mesures préventives :

Je vérifierai en amont les adresses I²C de chaque composant et m'assurerai qu'elles sont toutes différentes. J'utiliserai un scanner I²C (programme Arduino simple) pour détecter tous les appareils connectés sur le bus et confirmer qu'ils sont bien reconnus avant d'aller plus loin dans le développement.

1.4.6 Risque 6

Perte de données ou corruption du code source

Niveau de risque : Faible

Une panne informatique, une fausse manipulation ou un problème de synchronisation pourrait entraîner la perte d'une partie du code ou des documents rédigés, avec un impact potentiellement important sur l'avancement du projet.

Mesures préventives :

Le code sera versionné et poussé sur GitHub quotidiennement, conformément aux exigences du cahier des charges. Les documents seront sauvegardés régulièrement sur le réseau du CPNV. Ces deux pratiques combinées réduisent ce risque à un niveau très faible.

1.4.7 Risque 7

Difficultés avec l'environnement CLion / PlatformIO

Niveau de risque : Faible

CLion couplé à PlatformIO est un environnement plus avancé que l'IDE Arduino standard. Une configuration incorrecte, un problème de détection du port série ou une mise à jour inattendue pourrait ralentir le démarrage du développement.

Mesures préventives :

Je consacrerai le temps nécessaire à la mise en place et à la vérification de l'environnement de développement dès le premier jour, avant toute autre tâche de programmation. Si des difficultés persistantes apparaissent, je pourrai me rabattre temporairement sur l'IDE Arduino classique pour ne pas bloquer l'avancement, puis revenir à CLion une fois le problème résolu.

1.5 Planification initiale

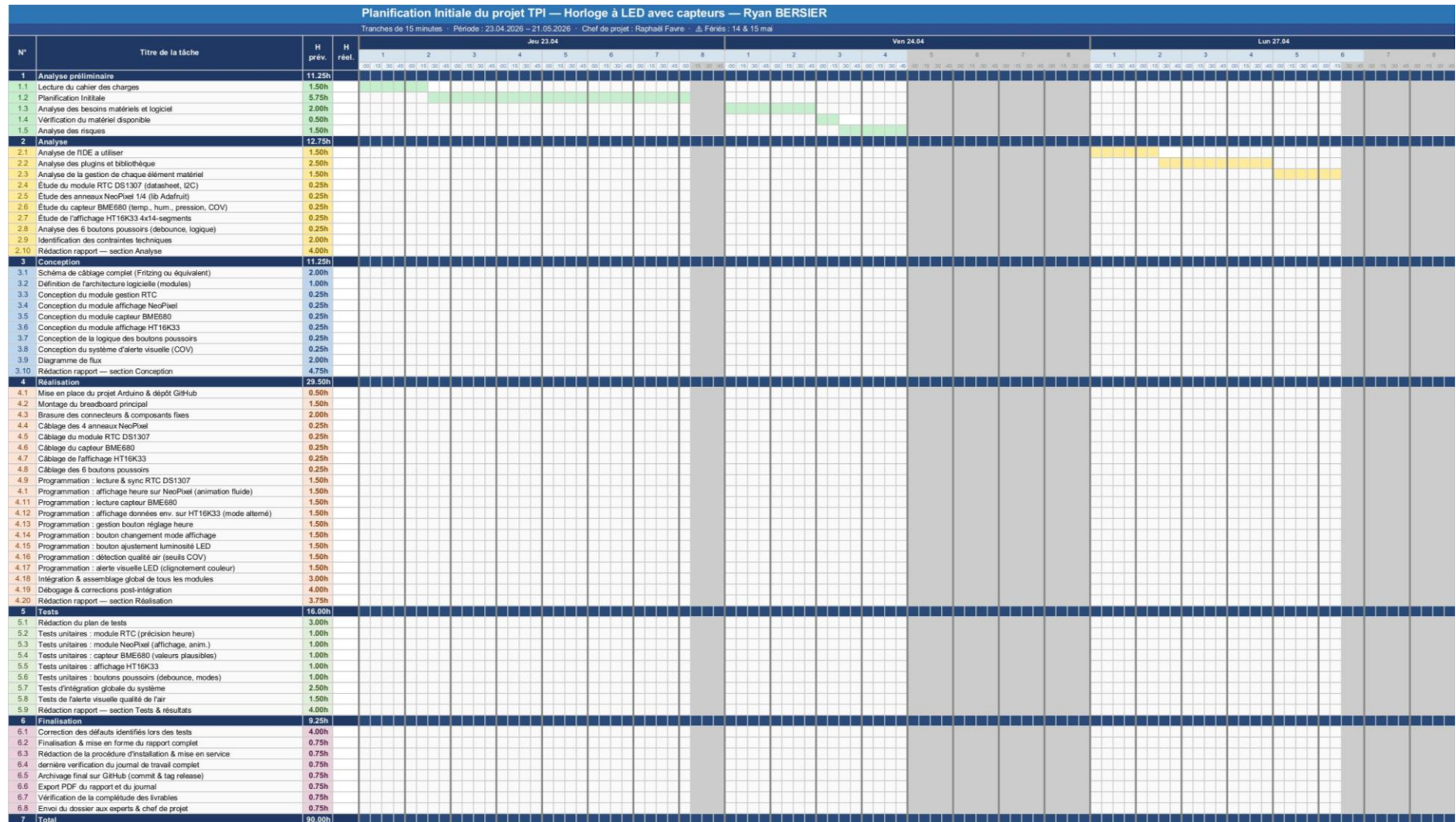


Figure 14 planning initial partie 1

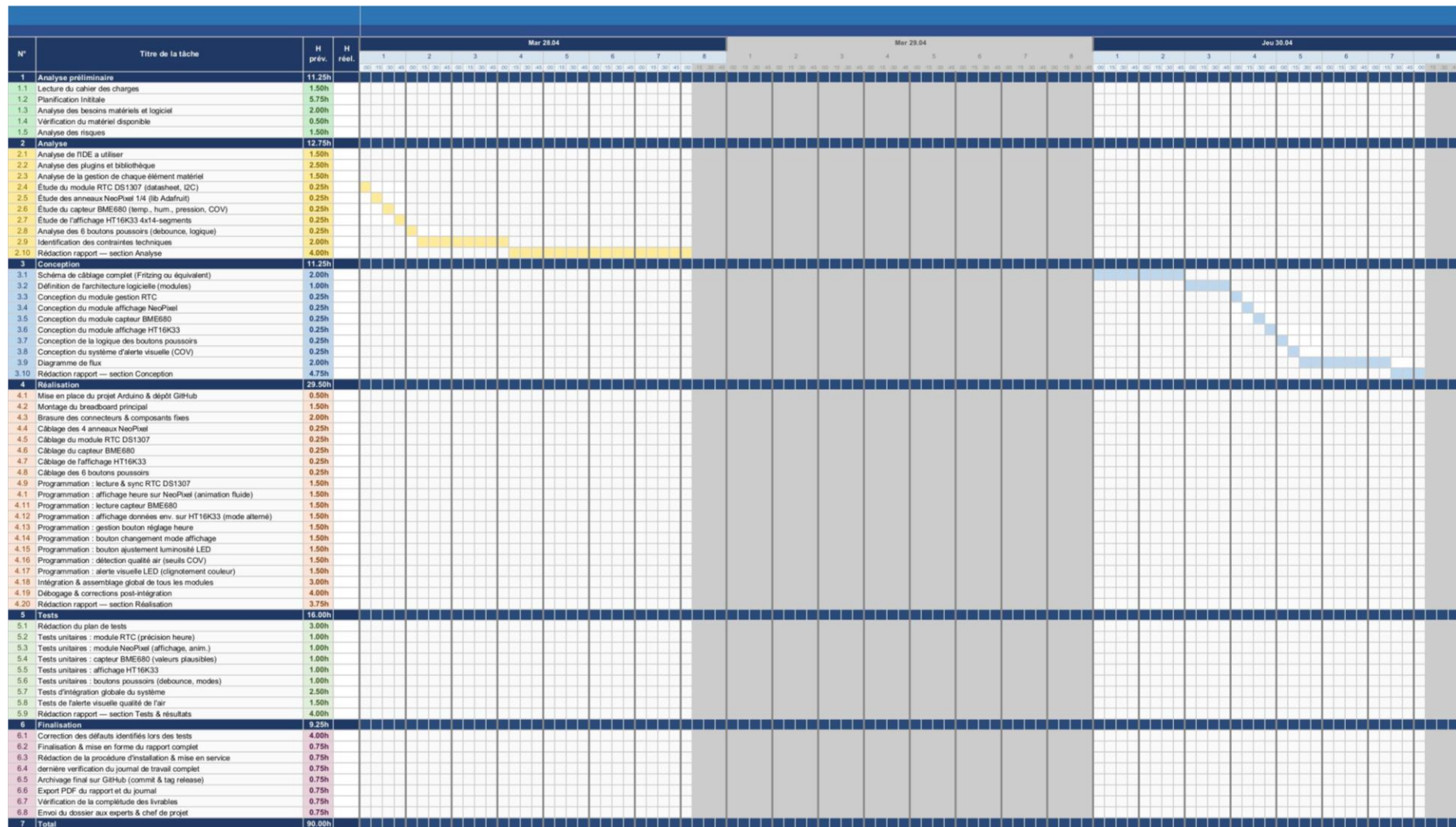


Figure 15 planning initial partie 2

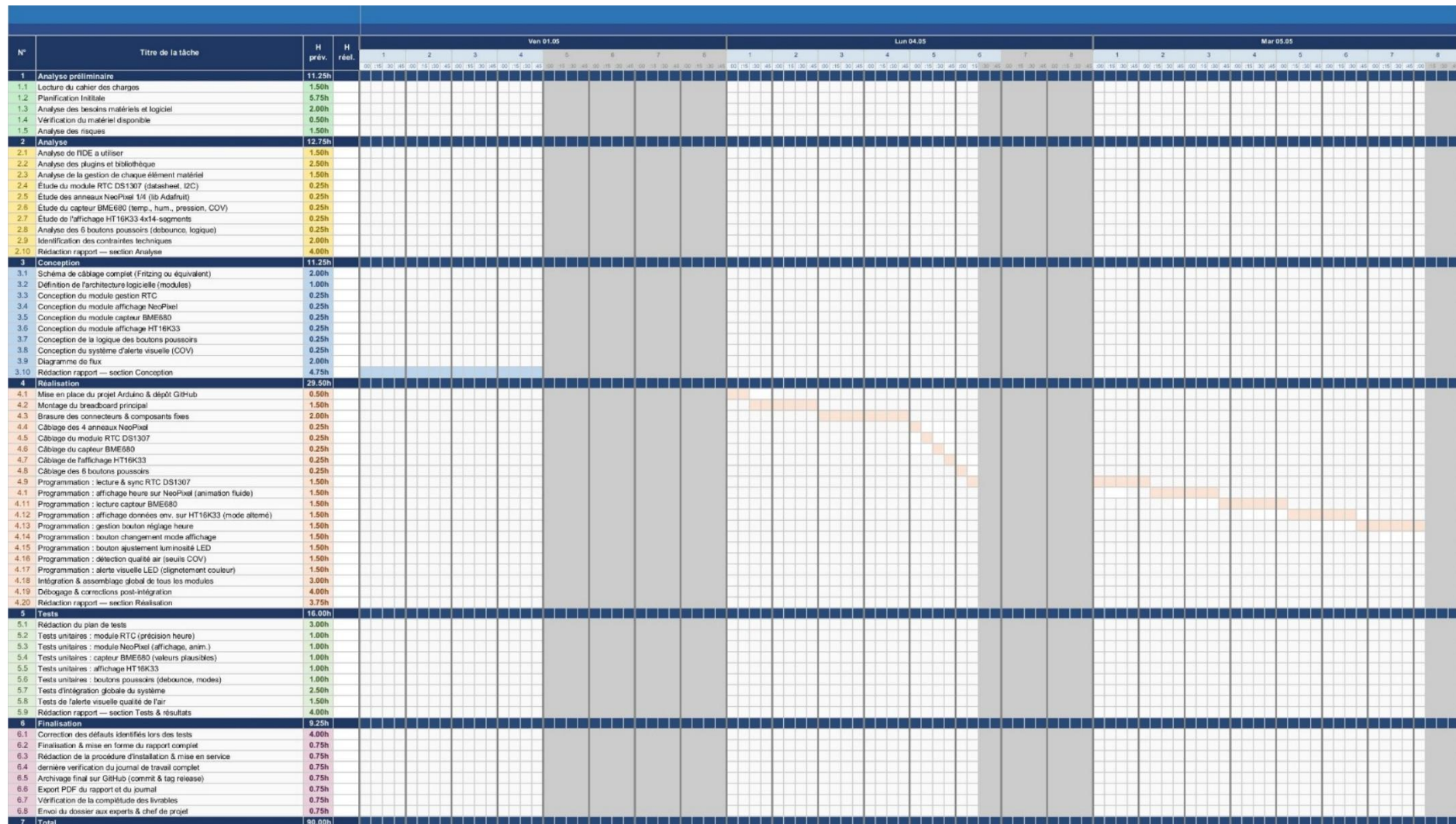


Figure 16 planning initial partie 3

N°	Titre de la tâche	H prév.	H réel.	Mer 06.05								Jeu 07.05								Ven 08.05							
				1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
				06:00-07:00	07:00-08:00	08:00-09:00	09:00-10:00	10:00-11:00	11:00-12:00	12:00-13:00	13:00-14:00	14:00-15:00	15:00-16:00	16:00-17:00	17:00-18:00	18:00-19:00	19:00-20:00	20:00-21:00	21:00-22:00	06:00-07:00	07:00-08:00	08:00-09:00	09:00-10:00	10:00-11:00	11:00-12:00	12:00-13:00	13:00-14:00
1	Analyse préliminaire	11.25h																									
1.1	Lecture du cahier des charges	1.50h																									
1.2	Planification initiale	5.75h																									
1.3	Analyse des besoins matériels et logiciel	2.00h																									
1.4	Vérification du matériel disponible	0.50h																									
1.5	Analyse des risques	1.50h																									
2	Analyse	12.75h																									
2.1	Analyse de l'IDE à utiliser	1.50h																									
2.2	Analyse des plugins et bibliothèque	2.50h																									
2.3	Analyse de la gestion de chaque élément matériel	1.50h																									
2.4	Étude du module RTC DS1307 (datasheet, I2C)	0.25h																									
2.5	Étude des anneaux NeoPixel 1/4 (lib Adafruit)	0.25h																									
2.6	Étude du capteur BME680 (temp., hum., pression, COV)	0.25h																									
2.7	Étude de l'affichage HT16K33 4x14-segments	0.25h																									
2.8	Analyse des 6 boutons poussoirs (debounce, logique)	0.25h																									
2.9	Identification des contraintes techniques	2.00h																									
2.10	Rédaction rapport — section Analyse	4.00h																									
3	Conception	11.25h																									
3.1	Schéma de câblage complet (Fritzing ou équivalent)	2.00h																									
3.2	Définition de l'architecture logicielle (modules)	1.00h																									
3.3	Conception du module gestion RTC	0.25h																									
3.4	Conception du module affichage NeoPixel	0.25h																									
3.5	Conception du module capteur BME680	0.25h																									
3.6	Conception du module affichage HT16K33	0.25h																									
3.7	Conception de la logique des boutons poussoirs	0.25h																									
3.8	Conception du système d'alerte visuelle (COV)	0.25h																									
3.9	Diagramme de flux	2.00h																									
3.10	Rédaction rapport — section Conception	4.75h																									
4	Réalisation	29.50h																									
4.1	Mise en place du projet Arduino & dépôt GitHub	0.50h																									
4.2	Montage du breadboard principal	1.50h																									
4.3	Brasage des connecteurs & composants fixes	2.00h																									
4.4	Câblage des 4 anneaux NeoPixel	0.25h																									
4.5	Câblage du module RTC DS1307	0.25h																									
4.6	Câblage du capteur BME680	0.25h																									
4.7	Câblage de l'affichage HT16K33	0.25h																									
4.8	Câblage des 6 boutons poussoirs	0.25h																									
4.9	Programmation : lecture & sync RTC DS1307	1.50h																									
4.10	Programmation : affichage heure sur NeoPixel (animation fluide)	1.50h																									
4.11	Programmation : lecture capteur BME680	1.50h																									
4.12	Programmation : affichage données env. sur HT16K33 (mode alterné)	1.50h																									
4.13	Programmation : gestion bouton réglage heure	1.50h																									
4.14	Programmation : bouton changement mode affichage	1.50h																									
4.15	Programmation : bouton ajustement luminosité LED	1.50h																									
4.16	Programmation : détection qualité air (seuils COV)	1.50h																									
4.17	Programmation : alerte visuelle LED (clignotement couleur)	1.50h																									
4.18	Intégration & assemblage global de tous les modules	3.00h																									
4.19	Débogage & corrections post-intégration	4.00h																									
4.20	Rédaction rapport — section Réalisation	3.75h																									
5	Tests	16.00h																									
5.1	Rédaction du plan de tests	3.00h																									
5.2	Tests unitaires : module RTC (précision heure)	1.00h																									
5.3	Tests unitaires : module NeoPixel (affichage, anim.)	1.00h																									
5.4	Tests unitaires : capteur BME680 (valeurs plausibles)	1.00h																									
5.5	Tests unitaires : affichage HT16K33	1.00h																									
5.6	Tests unitaires : boutons poussoirs (debounce, modes)	1.00h																									
5.7	Tests d'intégration globale du système	2.50h																									
5.8	Tests de l'alerte visuelle qualité de l'air	1.50h																									
5.9	Rédaction rapport — section Tests & résultats	4.00h																									
6	Finalisation	9.25h																									
6.1	Correction des défauts identifiés lors des tests	4.00h																									
6.2	Finalisation & mise en forme du rapport complet	0.75h																									
6.3	Rédaction de la procédure d'installation & mise en service	0.75h																									
6.4	dernière vérification du journal de travail complet	0.75h																									
6.5	Archivage final sur GitHub (commit & tag release)	0.75h																									
6.6	Export PDF du rapport et du journal	0.75h																									
6.7	Vérification de la complétude des livrables	0.75h																									
6.8	Envoi du dossier aux experts & chef de projet	0.75h																									
7	Total	90.00h																									

Figure 17 planning initial partie 4

N°	Titre de la tâche	H prév.	H réel.	Lun 11.05								Mar 12.05								Mer 13.05							
				1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
				08h	09h	10h	11h	12h	13h	14h	15h	08h	09h	10h	11h	12h	13h	14h	15h	08h	09h	10h	11h	12h	13h	14h	15h
1	Analyse préliminaire	11.25h																									
1.1	Lecture du cahier des charges	1.50h																									
1.2	Planification initiale	5.75h																									
1.3	Analyse des besoins matériels et logiciel	2.00h																									
1.4	Vérification du matériel disponible	0.50h																									
1.5	Analyse des risques	1.50h																									
2	Analyse	12.75h																									
2.1	Analyse de l'IDE à utiliser	1.50h																									
2.2	Analyse des plugins et bibliothèque	2.50h																									
2.3	Analyse de la gestion de chaque élément matériel	1.50h																									
2.4	Étude du module RTC DS1307 (datasheet, I2C)	0.25h																									
2.5	Étude des anneaux NeoPixel 14 (lib Adafruit)	0.25h																									
2.6	Étude du capteur BME680 (temp., hum., pression, COV)	0.25h																									
2.7	Étude de l'affichage HT16K33 4x14-segments	0.25h																									
2.8	Analyse des 6 boutons poussoirs (debounce, logique)	0.25h																									
2.9	Identification des contraintes techniques	2.00h																									
2.10	Rédaction rapport — section Analyse	4.00h																									
3	Conception	11.25h																									
3.1	Schéma de câblage complet (Fritzing ou équivalent)	2.00h																									
3.2	Définition de l'architecture logicielle (modules)	1.00h																									
3.3	Conception du module gestion RTC	0.25h																									
3.4	Conception du module affichage NeoPixel	0.25h																									
3.5	Conception du module capteur BME680	0.25h																									
3.6	Conception du module affichage HT16K33	0.25h																									
3.7	Conception de la logique des boutons poussoirs	0.25h																									
3.8	Conception du système d'alerte visuelle (COV)	0.25h																									
3.9	Diagramme de flux	2.00h																									
3.10	Rédaction rapport — section Conception	4.75h																									
4	Réalisation	29.50h																									
4.1	Mise en place du projet Arduino & dépôt GitHub	0.50h																									
4.2	Montage du breadboard principal	1.50h																									
4.3	Brasage des connecteurs & composants fixes	2.00h																									
4.4	Câblage des 4 anneaux NeoPixel	0.25h																									
4.5	Câblage du module RTC DS1307	0.25h																									
4.6	Câblage du capteur BME680	0.25h																									
4.7	Câblage de l'affichage HT16K33	0.25h																									
4.8	Câblage des 6 boutons poussoirs	0.25h																									
4.9	Programmation : lecture & sync RTC DS1307	1.50h																									
4.11	Programmation : affichage heure sur NeoPixel (animation fluide)	1.50h																									
4.12	Programmation : lecture capteur BME680	1.50h																									
4.13	Programmation : affichage données env. sur HT16K33 (mode alterné)	1.50h																									
4.14	Programmation : gestion bouton réglage heure	1.50h																									
4.15	Programmation : bouton changement mode affichage	1.50h																									
4.16	Programmation : bouton ajustement luminosité LED	1.50h																									
4.17	Programmation : détection qualité air (seuils COV)	1.50h																									
4.18	Programmation : alerte visuelle LED (clignotement couleur)	1.50h																									
4.19	Intégration & assemblage global de tous les modules	3.00h																									
4.20	Débugage & corrections post-intégration	4.00h																									
4.20	Rédaction rapport — section Réalisation	3.75h																									
5	Tests	16.00h																									
5.1	Rédaction du plan de tests	3.00h																									
5.2	Tests unitaires : module RTC (précision heure)	1.00h																									
5.3	Tests unitaires : module NeoPixel (affichage, anim.)	1.00h																									
5.4	Tests unitaires : capteur BME680 (valeurs plausibles)	1.00h																									
5.5	Tests unitaires : affichage HT16K33	1.00h																									
5.6	Tests unitaires : boutons poussoirs (debounce, modes)	1.00h																									
5.7	Tests d'intégration globale du système	2.50h																									
5.8	Tests de l'alerte visuelle qualité de l'air	1.50h																									
5.9	Rédaction rapport — section Tests & résultats	4.00h																									
6	Finalisation	9.25h																									
6.1	Correction des défauts identifiés lors des tests	4.00h																									
6.2	Finalisation & mise en forme du rapport complet	0.75h																									
6.3	Rédaction de la procédure d'installation & mise en service	0.75h																									
6.4	dernière vérification du journal de travail complet	0.75h																									
6.5	Archivage final sur GitHub (commit & tag release)	0.75h																									
6.6	Export PDF du rapport et du journal	0.75h																									
6.7	Vérification de la complétude des livrables	0.75h																									
6.8	Envoi du dossier aux experts & chef de projet	0.75h																									
7	Total	90.00h																									

Figure 18 planning initial partie 5

N°	Titre de la tâche	H prév.	H réel.	Jeu 14.05								Ven 15.05								Lun 18.05							
				1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
1	Analyse préliminaire	11.25h																									
1.1	Lecture du cahier des charges	1.50h																									
1.2	Planification initiale	5.75h																									
1.3	Analyse des besoins matériels et logiciel	2.00h																									
1.4	Vérification du matériel disponible	0.50h																									
1.5	Analyse des risques	1.50h																									
2	Analyse	12.75h																									
2.1	Analyse de l'IDE à utiliser	1.50h																									
2.2	Analyse des plugins et bibliothèques	2.50h																									
2.3	Analyse de la gestion de chaque élément matériel	1.50h																									
2.4	Étude du module RTC DS1307 (datasheet, I2C)	0.25h																									
2.5	Étude des anneaux NeoPixel 1/4 (lib Adafruit)	0.25h																									
2.6	Étude du capteur BME680 (temp., hum., pression, COV)	0.25h																									
2.7	Étude de l'affichage HT16K33 4x14-segments	0.25h																									
2.8	Analyse des 6 boutons poussoirs (debounce, logique)	0.25h																									
2.9	Identification des contraintes techniques	2.00h																									
2.10	Rédaction rapport — section Analyse	4.00h																									
3	Conception	14.25h																									
3.1	Schéma de câblage complet (Fritzing ou équivalent)	2.00h																									
3.2	Définition de l'architecture logicielle (modules)	1.00h																									
3.3	Conception du module gestion RTC	0.25h																									
3.4	Conception du module affichage NeoPixel	0.25h																									
3.5	Conception du module capteur BME680	0.25h																									
3.6	Conception du module affichage HT16K33	0.25h																									
3.7	Conception de la logique des boutons poussoirs	0.25h																									
3.8	Conception du système d'alerte visuelle (COV)	0.25h																									
3.9	Diagramme de flux	2.00h																									
3.10	Rédaction rapport — section Conception	4.75h																									
4	Réalisation	29.50h																									
4.1	Mise en place du projet Arduino & dépôt GitHub	0.50h																									
4.2	Montage du breadboard principal	1.50h																									
4.3	Brasage des connecteurs & composants fixes	2.00h																									
4.4	Câblage des 4 anneaux NeoPixel	0.25h																									
4.5	Câblage du module RTC DS1307	0.25h																									
4.6	Câblage du capteur BME680	0.25h																									
4.7	Câblage de l'affichage HT16K33	0.25h																									
4.8	Câblage des 6 boutons poussoirs	0.25h																									
4.9	Programmation : lecture & sync RTC DS1307	1.50h																									
4.1	Programmation : affichage heure sur NeoPixel (animation fluide)	1.50h																									
4.11	Programmation : lecture capteur BME680	1.50h																									
4.12	Programmation : affichage données env. sur HT16K33 (mode alterné)	1.50h																									
4.13	Programmation : gestion bouton réglage heure	1.50h																									
4.14	Programmation : bouton changement mode affichage	1.50h																									
4.15	Programmation : bouton ajustement luminosité LED	1.50h																									
4.16	Programmation : détection qualité air (seuls COV)	1.50h																									
4.17	Programmation : alerte visuelle LED (diploement couleur)	1.50h																									
4.18	Intégration & assemblage global de tous les modules	3.00h																									
4.19	Débugage & corrections post-intégration	4.00h																									
4.20	Rédaction rapport — section Réalisation	3.75h																									
5	Tests	16.00h																									
5.1	Rédaction du plan de tests	3.00h																									
5.2	Tests unitaires : module RTC (précision heure)	1.00h																									
5.3	Tests unitaires : module NeoPixel (affichage, anim.)	1.00h																									
5.4	Tests unitaires : capteur BME680 (valeurs plausibles)	1.00h																									
5.5	Tests unitaires : affichage HT16K33	1.00h																									
5.6	Tests unitaires : boutons poussoirs (debounce, modes)	1.00h																									
5.7	Tests d'intégration globale du système	2.50h																									
5.8	Tests de l'alerte visuelle qualité de l'air	1.50h																									
5.9	Rédaction rapport — section Tests & résultats	4.00h																									
6	Finalisation	9.25h																									
6.1	Correction des défauts identifiés lors des tests	4.00h																									
6.2	Finalisation & mise en forme du rapport complet	0.75h																									
6.3	Rédaction de la procédure d'installation & mise en service	0.75h																									
6.4	dernière vérification du journal de travail complet	0.75h																									
6.5	Archivage final sur GitHub (commit & tag release)	0.75h																									
6.6	Export PDF du rapport et du journal	0.75h																									
6.7	Vérification de la complétude des livrables	0.75h																									
6.8	Envoi du dossier aux experts & chef de projet	0.75h																									
7	Total	96.00h																									

Figure 19 planning initial partie 6

N°	Titre de la tâche	H prév.	H réel.	Mar 19.05								Mer 20.05								Jeu 21.05							
				1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
1	Analyse préliminaire	11.25h																									
1.1	Lecture du cahier des charges	1.50h																									
1.2	Planification initiale	5.75h																									
1.3	Analyse des besoins matériels et logiciel	2.00h																									
1.4	Vérification du matériel disponible	0.50h																									
1.5	Analyse des risques	1.50h																									
2	Analyse	12.75h																									
2.1	Analyse de l'IDE à utiliser	1.50h																									
2.2	Analyse des plug-ins et bibliothèque	2.50h																									
2.3	Analyse de la gestion de chaque élément matériel	1.50h																									
2.4	Étude du module RTC DS1307 (datasheet, I2C)	0.25h																									
2.5	Étude des anneaux NeoPixel 1/4 (lib Adafruit)	0.25h																									
2.6	Étude du capteur BME680 (temp., hum., pression, COV)	0.25h																									
2.7	Étude de l'affichage HT16K33 4x14-segments	0.25h																									
2.8	Analyse des 6 boutons poussoirs (debounce, logique)	0.25h																									
2.9	Identification des contraintes techniques	2.00h																									
2.10	Rédaction rapport — section Analyse	4.00h																									
3	Conception	12.25h																									
3.1	Schéma de câblage complet (Fritzing ou équivalent)	2.00h																									
3.2	Définition de l'architecture logicielle (modules)	1.00h																									
3.3	Conception du module gestion RTC	0.25h																									
3.4	Conception du module affichage NeoPixel	0.25h																									
3.5	Conception du module capteur BME680	0.25h																									
3.6	Conception du module affichage HT16K33	0.25h																									
3.7	Conception de la logique des boutons poussoirs	0.25h																									
3.8	Conception du système d'alerte visuelle (COV)	0.25h																									
3.9	Diagramme de flux	2.00h																									
3.10	Rédaction rapport — section Conception	4.75h																									
4	Réalisation	29.50h																									
4.1	Mise en place du projet Arduino & dépôt GitHub	0.50h																									
4.2	Montage du breadboard principal	1.50h																									
4.3	Brasage des connecteurs & composants fixes	2.00h																									
4.4	Câblage des 4 anneaux NeoPixel	0.25h																									
4.5	Câblage du module RTC DS1307	0.25h																									
4.6	Câblage du capteur BME680	0.25h																									
4.7	Câblage de l'affichage HT16K33	0.25h																									
4.8	Câblage des 6 boutons poussoirs	0.25h																									
4.9	Programmation : lecture & sync RTC DS1307	1.50h																									
4.11	Programmation : affichage heure sur NeoPixel (animation fluide)	1.50h																									
4.11	Programmation : lecture capteur BME680	1.50h																									
4.12	Programmation : affichage données env. sur HT16K33 (mode alterné)	1.50h																									
4.13	Programmation : gestion bouton réglage heure	1.50h																									
4.14	Programmation : bouton changement mode affichage	1.50h																									
4.15	Programmation : bouton ajustement luminosité LED	1.50h																									
4.16	Programmation : détection qualité air (seuils COV)	1.50h																									
4.17	Programmation : alerte visuelle LED (clignotement couleur)	1.50h																									
4.18	Intégration & assemblage global de tous les modules	3.00h																									
4.19	Débugage & corrections post-intégration	4.00h																									
4.20	Rédaction rapport — section Réalisation	3.75h																									
5	Tests	16.00h																									
5.1	Rédaction du plan de tests	3.00h																									
5.2	Tests unitaires : module RTC (précision heure)	1.00h																									
5.3	Tests unitaires : module NeoPixel (affichage, anim.)	1.00h																									
5.4	Tests unitaires : capteur BME680 (valeurs plausibles)	1.00h																									
5.5	Tests unitaires : affichage HT16K33	1.00h																									
5.6	Tests unitaires : boutons poussoirs (debounce, modes)	1.00h																									
5.7	Tests d'intégration globale du système	2.50h																									
5.8	Tests de l'alerte visuelle qualité de l'air	1.50h																									
5.9	Rédaction rapport — section Tests & résultats	4.00h																									
6	Finalisation	9.25h																									
6.1	Correction des défauts identifiés lors des tests	4.00h																									
6.2	Finalisation & mise en forme du rapport complet	0.75h																									
6.3	Rédaction de la procédure d'installation & mise en service	0.75h																									
6.4	dernière vérification du journal de travail complet	0.75h																									
6.5	Archivage final sur GitHub (commit & tag release)	0.75h																									
6.6	Export PDF du rapport et du journal	0.75h																									
6.7	Vérification de la complétude des livrables	0.75h																									
6.8	Envoi du dossier aux experts & chef de projet	0.75h																									
7	Total	90.00h																									

Figure 20 planning initial partie 7

2 Analyse

2.1 Justification choix de l'IDE

Critère	Arduino IDE	Visual Studio Code	CLion + PlatformIO
Autocomplétion	Très basique, peu fiable	Bonne via extension C/C++	Excellente, moteur JetBrains (très précis sur le code embarqué)
Navigation dans le code	Absente	Correcte	Avancée aller à la définition, recherche de références, refactoring
Gestion des bibliothèques	Manuelle via gestionnaire basique	Via PlatformIO (si installé)	Via PlatformIO intégré nativement gestion des dépendances automatique
Structure du projet	Fichier unique .ino difficile à organiser	Multi-fichiers possible	Multi-fichiers et multi-modules natif, idéal pour séparer les composants
Débogage	Aucun débogage réel	Limité	Débogage avancé intégré (breakpoints, inspection de variables)
Intégration Git	Absente	Basique via extension	Native et complète dans l'IDE
Détection d'erreurs	À la compilation seulement	Partielle en temps réel	En temps réel, avec suggestions de correction immédiates
Support I²C / librairies Adafruit	Oui	Oui via PlatformIO	Oui PlatformIO gère toutes les libs Adafruit sans configuration manuelle
Qualité de l'interface	Très simple, peu ergonomique	Bonne mais nécessite beaucoup de configuration	Professionnelle, tout est prêt à l'emploi sans configuration complexe
Adapté aux projets modulaires	Non	Partiellement	Oui conçu pour les projets découpés en modules/fichiers
Familiarité personnelle	Connue mais limitante	Connue mais légère pour l'embarquer	Choisie pour monter en compétence sur un outil professionnel

2.2 Analyse des éléments électronique

L'analyse comparative présentée dans les sections suivantes s'inscrit dans un contexte purement hypothétique. Dans la réalité, l'ensemble du matériel utilisé dans ce projet est fourni par le CPNV et ne fait l'objet d'aucun choix de ma part. Cependant, afin de démontrer ma capacité à évaluer et justifier des choix techniques, j'ai imaginé un scénario dans lequel je serais libre de sélectionner chaque composant moi-même, avec un budget plus conséquent permettant d'envisager des alternatives parfois plus coûteuses que celles mises à disposition. Les justifications apportées sont donc purement techniques et ne reflètent en aucun cas une critique du matériel fourni, qui reste tout à fait adapté à la réalisation de ce projet.

2.2.1 Carte Arduino UNO REV3

2.2.1.1 Rôle dans le projet

La carte Arduino UNO REV3 est le microcontrôleur central du projet. Elle exécute le programme principal en temps réel : elle interroge le module RTC pour connaître l'heure, calcule la position des LED à allumer sur l'anneau NeoPixel, lit les valeurs du capteur BME680, envoie les données à afficher sur l'écran HT16K33, et surveille en permanence l'état des six boutons poussoirs. Toute la logique du système repose sur elle.

2.2.1.2 Alternative : Raspberry Pi Pico (RP2040)

Le Raspberry Pi Pico est un microcontrôleur moderne basé sur le chip RP2040, dual-core à 133 MHz, avec 264 Ko de RAM et un support natif du MicroPython et du C/C++. Il est nettement plus puissant que l'Arduino UNO, dispose de deux cœurs permettant de paralléliser les tâches, et intègre du matériel PIO (Programmable I/O) idéal pour piloter des LED NeoPixel sans surcharger le processeur.

2.2.1.3 Meilleur choix : Raspberry Pi Pico

Dans un monde où l'on choisit librement ses composants, le Raspberry Pi Pico serait le meilleur choix. Sa puissance de calcul supérieure permettrait de gérer plus facilement les animations LED fluides, la lecture des capteurs et la gestion des boutons simultanément, sans risquer de saturer le processeur. L'Arduino UNO REV3 reste un excellent outil pédagogique, mais ses 2 Ko de RAM et son unique cœur à 16 MHz montrent leurs limites sur un projet aussi multifonction.

2.2.2 Anneaux Adafruit NeoPixel 1/4 60 LED au total (×4)

2.2.2.1 Rôle dans le projet

Les quatre quarts d'anneau NeoPixel forment, une fois assemblés, un cercle complet de 60 LED RGB adressables individuellement. Dans ce projet, ils servent d'affichage principal de l'horloge : les LED représentent visuellement les heures, les minutes et les secondes par des animations lumineuses. Ils jouent également le rôle d'indicateur d'alerte environnementale en changeant de couleur ou de comportement lorsque la qualité de l'air se dégrade. Un seul fil de données suffit pour piloter l'ensemble des 60 LED en cascade.

2.2.2.2 Alternative : Anneau LED WS2812B (60 LED d'un seul tenant)

Il existe des anneaux LED complets de 60 LED WS2812B (la puce utilisée dans les NeoPixel) en un seul module circulaire. Ils reposent sur le même protocole de communication et sont compatibles avec la bibliothèque Adafruit NeoPixel, mais se présentent sous forme d'un anneau monobloc au lieu de quatre quarts séparés.

2.2.2.3 Meilleur choix : Anneau WS2812B 60 LED monobloc

Un anneau monobloc serait légèrement préférable en termes de montage : il n'y a aucun joint à souder entre les quarts, ce qui élimine les risques de mauvais contact aux jonctions et simplifie l'assemblage mécanique. Sur le plan électronique et logiciel, les deux solutions sont strictement équivalentes. Le choix des quatre quarts Adafruit n'est donc pas le plus optimal mécaniquement, même s'il reste parfaitement fonctionnel.

2.2.3 Capteur environnemental BME680

2.2.3.1 Rôle dans le projet

Le BME680 est le capteur qui fournit toutes les données environnementales du système. Il mesure en continu la température ambiante, l'humidité relative, la pression atmosphérique et un indice de qualité de l'air basé sur la détection de composés organiques volatils (COV). Ces quatre valeurs sont ensuite affichées en alternance sur l'écran HT16K33, et le niveau de COV déclenche l'alerte visuelle sur l'anneau LED lorsqu'il dépasse un seuil critique. Il communique avec l'Arduino via le bus I²C.

2.2.3.2 Alternative : Capteur SCD41 (CO₂, température, humidité) de Sensirion

Le SCD41 est un capteur de CO₂ véritable (mesure photoacoustique NDIR), couplé à une mesure de température et d'humidité. Contrairement au BME680 qui estime la qualité de l'air de manière indirecte via les COV, le SCD41 mesure directement la concentration de CO₂ en ppm ce qui est précisément le problème identifié dans le cahier des charges (les boîtiers CO₂ des salles de classe).

2.2.3.3 Meilleur choix : SCD41

Dans le contexte de ce projet, dont l'objectif premier est d'alerter sur la mauvaise qualité de l'air en salle de classe un problème directement lié au CO₂ expiré par les occupants le SCD41 serait techniquement plus pertinent. Il mesure ce qu'on cherche vraiment à surveiller, là où le BME680 donne une estimation indirecte via les COV. Le BME680 a l'avantage de mesurer aussi la pression atmosphérique, mais pour ce cas d'usage précis, la mesure directe du CO₂ par le SCD41 est plus fiable et plus cohérente avec la problématique décrite dans le cahier des charges.

2.2.4 Module horloge temps réel RTC DS1307

2.2.4.1 Rôle dans le projet

Le module RTC DS1307 a pour unique but de maintenir l'heure exacte en permanence, indépendamment de l'Arduino. Contrairement à une simple fonction de comptage logiciel dans le microcontrôleur, ce module dispose de son propre oscillateur

à quartz et d'une pile de sauvegarde qui lui permet de continuer à compter le temps même lorsque l'appareil est hors tension. À chaque démarrage du système, l'Arduino lit l'heure courante depuis le DS1307 via I²C pour initialiser son affichage.

2.2.4.2 Alternative : Module RTC DS3231

Le DS3231 est l'évolution directe du DS1307. Il intègre un oscillateur à quartz compensé en température (TCXO), ce qui lui confère une précision nettement supérieure : une dérive de ± 2 minutes par an contre plusieurs minutes par mois pour le DS1307. Il intègre également un capteur de température interne et reste compatible avec le même protocole I²C et les mêmes bibliothèques.

2.2.4.3 Meilleur choix : DS3231

Le DS3231 est clairement le meilleur choix pour une horloge murale. Une horloge qui dérive de plusieurs minutes par mois perd rapidement sa crédibilité en tant qu'instrument de mesure du temps. Le DS3231, avec sa précision quasi parfaite sur une année entière, garantit que l'heure affichée reste juste sur le long terme sans nécessiter de recalibration fréquente. Pour un projet qui se veut une horloge fiable et autonome, cette précision accrue justifie pleinement le choix du DS3231.

2.2.5 Adafruit 4×14 segments alphanumérique HT16K33

2.2.5.1 Rôle dans le projet

L'affichage HT16K33 sert d'écran secondaire textuel pour présenter les données environnementales collectées par le BME680. Il affiche en alternance la température, l'humidité, la pression et l'indice de qualité de l'air sous forme de texte et de chiffres lisibles. Son format 14 segments par digit permet d'afficher non seulement des chiffres mais aussi des lettres, ce qui facilite l'identification de chaque mesure (par exemple afficher "TEMP" avant la valeur). Il communique via I²C, ce qui limite le nombre de broches utilisées sur l'Arduino.

2.2.5.2 Alternative : Petit écran OLED 128×64 pixels (SSD1306)

Un écran OLED monochrome de 0.96 pouce basé sur le contrôleur SSD1306 offre une surface d'affichage en pixels libres, permettant d'afficher du texte de taille variable, des icônes, des graphiques ou plusieurs valeurs simultanément sur le même écran. Il communique également en I²C et consomme très peu d'énergie.

2.2.5.3 Meilleur choix : Écran OLED SSD1306

L'écran OLED serait un meilleur choix dans un projet idéal. Il permet d'afficher toutes les mesures environnementales simultanément sur un seul écran sans avoir recours à l'alternance, d'ajouter des unités claires (°C, %, hPa), de créer une mise en page plus lisible et intuitive, et même d'afficher des indicateurs visuels comme une jauge de qualité de l'air. L'affichage 14 segments reste lisible et élégant, mais il est limité en termes d'information affichable en une seule fois, ce qui oblige à faire défiler les mesures et peut rendre la lecture moins immédiate.

2.2.6 Boutons poussoirs (×6)

2.2.6.1 Rôle dans le projet

Les six boutons poussoirs constituent l'interface de contrôle manuelle du système. Ils permettent à l'utilisateur de régler l'heure manuellement sur le module RTC, de faire défiler les différents modes d'affichage sur l'écran alphanumérique, et d'ajuster la luminosité de l'anneau LED. Chaque bouton est relié à une broche numérique de l'Arduino, et une gestion logicielle du rebond (debounce) est nécessaire pour éviter les détections multiples lors d'une seule pression.

2.2.6.2 Alternative : Encodeur rotatif avec bouton intégré (KY-040)

Un encodeur rotatif est un composant qui détecte la rotation dans un sens ou dans l'autre, en plus d'un appui central. Avec un ou deux encodeurs, il serait possible de remplacer plusieurs boutons : tourner pour naviguer entre les valeurs ou les modes, et appuyer pour confirmer ou changer de paramètre. C'est l'interface que l'on retrouve sur la plupart des appareils électroniques modernes (amplificateurs, imprimantes 3D, thermostats).

2.2.6.3 Meilleur choix : Boutons poussoirs classiques

Pour ce projet précis, les boutons poussoirs classiques restent le meilleur choix. Les six fonctions à contrôler (réglage heure, modes d'affichage, luminosité) sont des actions distinctes et indépendantes qui se prêtent bien à des boutons dédiés. L'encodeur rotatif est plus élégant pour naviguer dans un menu, mais introduit une complexité logicielle supplémentaire (gestion de la rotation, de la vitesse, d'un menu structuré) qui n'est pas justifiée pour un nombre aussi limité de paramètres. Des boutons bien étiquetés offrent une interaction plus directe et intuitive dans ce contexte.

2.3 Contraintes techniques

Tout projet de développement s'inscrit dans un cadre de contraintes qui délimitent le champ des possibles et orientent les choix de conception. Ces contraintes ne sont pas des obstacles, mais des paramètres à intégrer dès le départ pour éviter les mauvaises surprises en cours de réalisation.

2.3.1 Contrainte 1

Limitation des ressources du microcontrôleur

L'Arduino UNO REV3 dispose de ressources matérielles très limitées : seulement 2 Ko de RAM, 32 Ko de mémoire Flash et un processeur ATmega328P cadencé à 16 MHz sur un seul cœur. Le projet doit faire cohabiter simultanément la gestion de l'anneau NeoPixel (60 LED), la lecture du capteur BME680, la communication avec le RTC DS1307, l'affichage sur le HT16K33 et la surveillance des six boutons poussoirs. Chaque bibliothèque utilisée consomme une part de cette mémoire limitée, et le code devra être optimisé pour ne pas dépasser les capacités de la carte, sous peine de comportements imprévisibles ou de plantages.

2.3.2 Contrainte 2

Partage du bus I²C entre plusieurs composants

Trois composants du projet — le BME680, le RTC DS1307 et l'affichage HT16K33 — communiquent tous via le même bus I²C, sur les broches SDA et SCL de l'Arduino. Chaque composant doit donc avoir une adresse I²C unique et bien configurée. Une collision d'adresses ou un problème de câblage sur ce bus unique rendrait plusieurs composants inaccessibles simultanément. Cette contrainte impose une vérification rigoureuse des adresses avant l'intégration et une gestion soigneuse du câblage de ces deux broches partagées.

2.3.3 Contrainte 3

Alimentation et consommation électrique

L'ensemble des composants est alimenté via la carte Arduino, elle-même alimentée par le câble USB ou une alimentation externe. L'anneau de 60 LED NeoPixel est le composant le plus gourmand en énergie : à pleine luminosité, chaque LED peut consommer jusqu'à 60 mA, ce qui représente théoriquement 3,6 A pour les 60 LED en blanc pur. Le port USB d'un ordinateur ne fournit généralement que 500 mA. Il sera donc impératif de limiter la luminosité maximale des LED dans le code afin de ne pas dépasser la capacité d'alimentation disponible et éviter tout risque de surchauffe ou de dommage matériel.

2.3.4 Contrainte 4

Gestion du temps réel et des priorités d'exécution

L'Arduino exécute le code de manière séquentielle sur un seul cœur, sans système d'exploitation ni gestion native des priorités. Or, le projet nécessite de gérer plusieurs tâches quasi simultanément : mettre à jour l'affichage LED, lire les capteurs, surveiller les boutons et rafraîchir l'écran alphanumérique. L'utilisation de la fonction `delay()` est donc à proscrire, car elle bloque l'exécution de tout le reste pendant sa durée. Le code devra reposer sur une logique de timing non-bloquante, basée sur `millis()`, pour que chaque tâche soit exécutée à sa propre fréquence sans bloquer les autres.

2.3.5 Contrainte 5

Gestion du rebond des boutons (debounce)

Les boutons poussoirs mécaniques génèrent naturellement des signaux parasites lors d'un appui ou d'un relâchement : le contact rebondit plusieurs fois en quelques millisecondes avant de se stabiliser. Sans traitement, l'Arduino peut interpréter un seul appui comme plusieurs pressions consécutives, rendant l'interaction utilisateur erratique et peu fiable. Une gestion logicielle du debounce devra être implémentée pour chacun des six boutons, en filtrant les changements d'état trop rapprochés dans le temps.

2.3.6 Contrainte 6

Précision et dérive de l'horloge

Le module RTC DS1307 utilisé dans ce projet présente une dérive pouvant atteindre plusieurs minutes par mois selon les conditions de température. Pour une horloge murale destinée à être consultée quotidiennement, cette dérive est acceptable à court terme mais peut devenir gênante sur la durée. Le système doit donc prévoir la possibilité pour l'utilisateur de recaler manuellement l'heure via les boutons poussoirs, afin de corriger toute dérive sans nécessiter de reprogrammation de la carte.

2.3.7 Contrainte 7

Fiabilité de la mesure environnementale

Le capteur BME680 mesure la qualité de l'air via un indice COV qui nécessite une phase de chauffe et de calibration lors de chaque mise sous tension. Durant les premières minutes de fonctionnement, les valeurs retournées peuvent être instables ou non représentatives de la réalité. Le code devra tenir compte de ce temps de stabilisation et éviter de déclencher de fausses alertes visuelles pendant cette période d'initialisation du capteur.

2.3.8 Contrainte 8

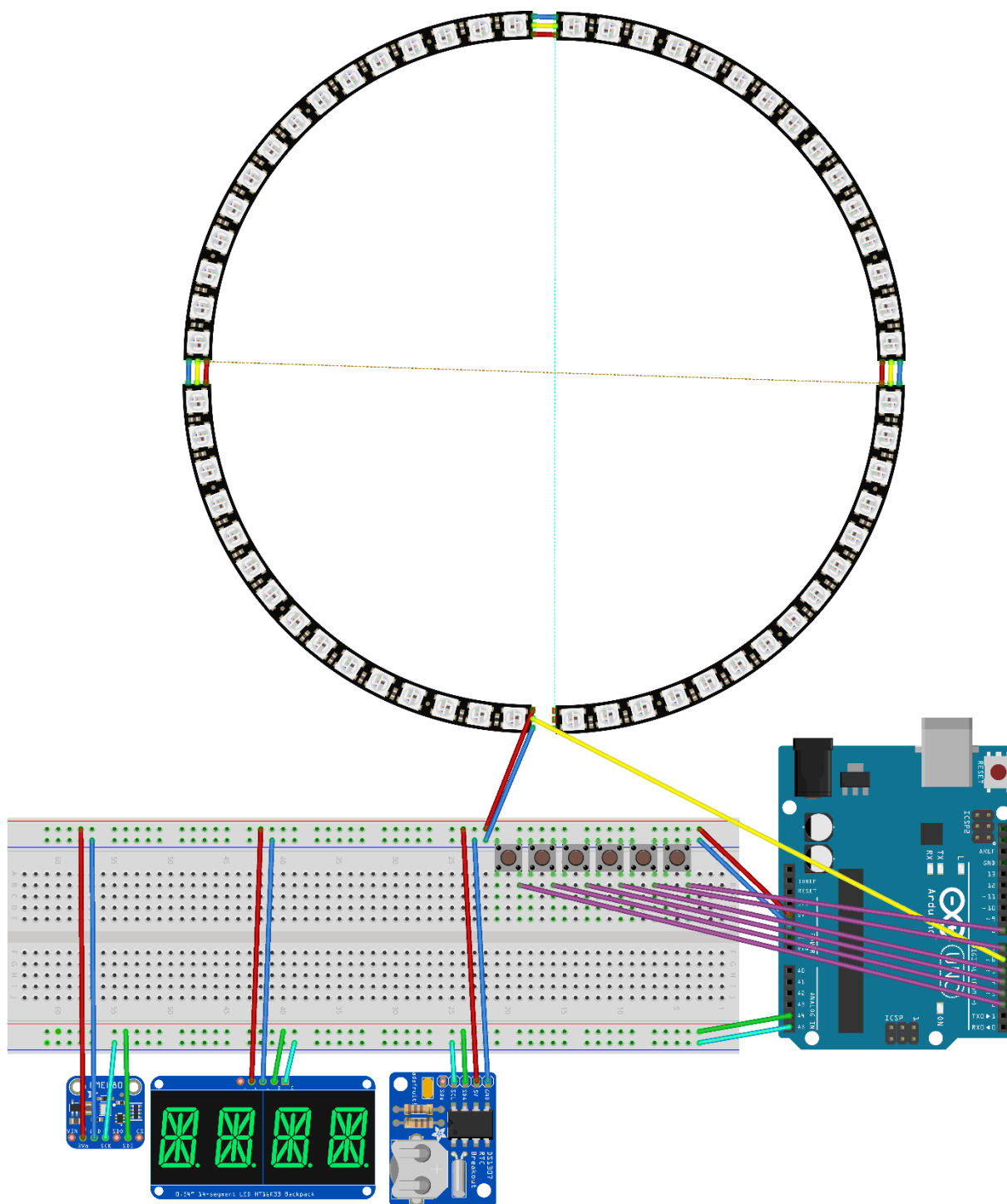
Contrainte mécanique et physique du montage

L'assemblage des quatre quarts d'anneau NeoPixel en un cercle complet impose une précision mécanique lors de la brasure des jonctions entre chaque quart. Un mauvais alignement ou une connexion défectueuse entre deux quarts peut interrompre la transmission du signal vers les LED situées en aval, rendant une partie de l'anneau non fonctionnelle. Le montage physique du projet dans son ensemble devra être soigné et solide pour garantir un fonctionnement stable dans le temps.

3 Conception

3.1 Schéma électronique

3.1.1 Platine d'essai



fritzing

Figure 21 Platine d'essai du câblage qui provient de Fritzing

3.1.2 Schématique du circuit électronique

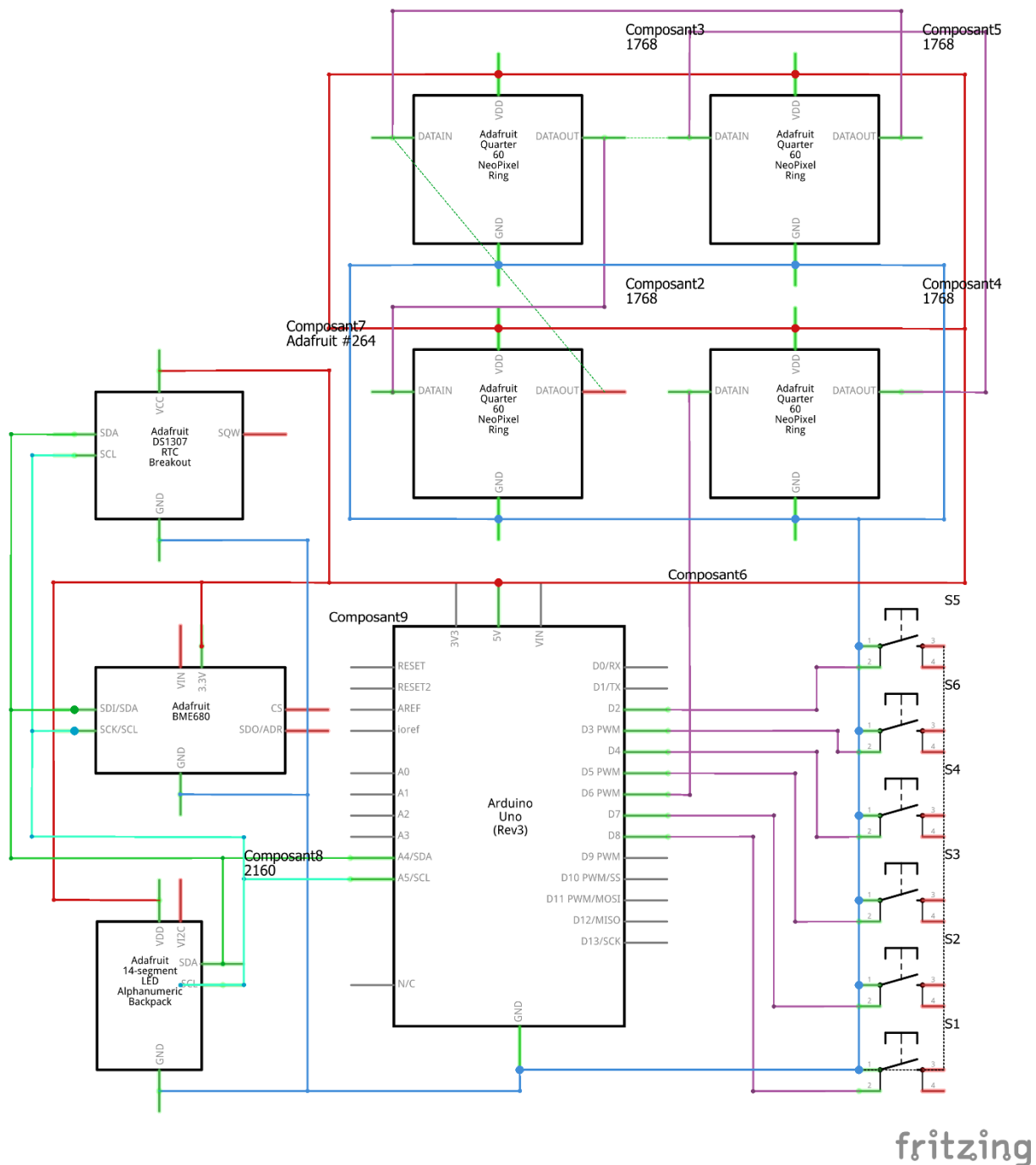


Figure 22 schématique du circuit électronique fait sur Fritzing

3.2 Définition de l'architecture logiciel

L'architecture logicielle du projet est organisée en modules fonctionnels simples, afin de garantir un code lisible, maintenable et facilement testable sur microcontrôleur Arduino.

Le programme repose sur une boucle principale non bloquante (loop) qui orchestre en continu cinq blocs métiers :

3.2.1 Acquisition des entrées utilisateur

Les 6 boutons (D2, D3, D4, D5, D7, D8) sont lus avec anti-rebond logiciel. Chaque appui génère un événement interprété selon le mode courant (normal ou configuration).

3.2.2 Gestion temporelle (RTC DS1307)

Le module RTC fournit l'heure et la date en temps réel. Ces données pilotent :

- L'affichage de l'horloge sur le ring NeoPixel (heures, minutes, secondes),
- Le mode configuration (édition jour/mois/heure/minute/seconde puis écriture RTC).

3.2.3 Acquisition environnementale (BME680)

Le capteur BME680 est lu périodiquement (température, humidité, pression, résistance gaz). La température est compensée (Tcomp) par un offset fixe pour corriger l'échauffement capteur. La valeur eCO2/PPM~ est estimée via un modèle logiciel :

- Filtrage du signal gaz,
- Baseline adaptative,
- Conversion non linéaire,
- Compensation légère température/humidité.

3.2.4 Gestion d'affichage (HT16K33 14 segments)

En mode normal, les informations s'affichent en rotation automatique toutes les 10 s : eCO2 -> TEMP -> HUMD -> ATMO -> DATE.

Un TAG est affiché avant chaque valeur pour améliorer la lisibilité. En mode configuration, l'affichage bascule sur le champ en cours d'édition.

3.2.5 Gestion visuelle de l'anneau NeoPixel

L'anneau NeoPixel de 60 LEDs représente une horloge analogique en temps réel :

- Heure : aiguille en rouge
- Minute : aiguille en vert
- Seconde : progression en bleu cumulatif de la LED 0 jusqu'à la seconde courante

Le rendu intègre des règles de superposition pour conserver une lecture claire en cas de recouvrement :

- Heure + Minute → jaune
- Heure + Seconde → magenta
- Minute + Seconde → cyan-vert
- Heure + Minute + Seconde → blanc

Des marqueurs d'heures (LED de repère) sont également affichés sur les positions multiples de 5, avec des teintes adaptées lorsqu'ils se superposent aux aiguilles.

En cas de dégradation de la qualité de l'air (eCO₂ au-dessus du seuil), l'anneau bascule en mode alerte :

- Clignotement rouge rapide (ON/OFF),
- Luminosité renforcée pour une meilleure visibilité,
- Possibilité de désactivation temporaire via bouton dédié,
- Réarmement automatique lorsque la qualité d'air repasse sous le seuil.

3.3 Conception des modules

La conception logicielle est découpée en modules cohérents, chacun responsable d'un sous-ensemble clair des fonctionnalités du système.

3.3.1 Module 1 Gestion des entrées (boutons)

Ce module lit les 6 boutons poussoirs avec une logique d'anti-rebond logiciel.

Il transforme les changements d'état physiques en événements d'appui exploitables par l'application (menu, incrémentation, décrémentation, changement d'affichage, luminosité, désactivation alerte).

3.3.2 Module 2 Gestion RTC (temps réel)

Ce module initialise et interroge le DS1307 via I2C.

Il fournit l'heure/date courante au reste du programme et applique les nouvelles valeurs validées en mode configuration (jour, mois, heure, minute, seconde).

3.3.3 Module 3 Gestion capteur environnemental (BME680)

Ce module réalise l'acquisition périodique des mesures (température, humidité, pression, gaz).

Il applique :

- une compensation de température (T_{comp}),
- un filtrage du signal gaz,
- une baseline adaptative,
- une conversion vers une estimation PPM~ / eCO₂.

Il intègre également un mécanisme de reprise en cas d'échec de lecture capteur.

3.3.4 Module 4 Gestion affichage alphanumérique (HT16K33)

Ce module pilote l'afficheur 4x14 segments.

En mode normal, il réalise une rotation automatique des pages (eCO2, TEMP, HUMD, ATMO, DATE) avec affichage d'un TAG avant la valeur.

En mode configuration, il affiche uniquement le champ en cours d'édition.

3.3.5 Module 5 Gestion anneau NeoPixel (horloge + alerte)

Ce module pilote l'anneau NeoPixel 60 LEDs et assure à la fois l'affichage horaire et la signalisation d'alerte.

En mode normal, il dessine une horloge analogique :

- Heure en rouge,
- Minute en vert,
- Secondes en bleu cumulatif (de la LED 0 jusqu'à la seconde courante).

Le module gère également les superpositions entre aiguilles (et marqueurs horaires) à l'aide de couleurs dédiées, afin de conserver une lecture visuelle claire lorsque plusieurs informations occupent la même LED.

En mode alerte qualité d'air (eCO2 au-dessus du seuil), le comportement bascule automatiquement en :

- Clignotement rouge rapide de l'anneau complet,
- Luminosité renforcée pour une visibilité maximale,
- Désactivation temporaire possible via bouton utilisateur,
- Réarmement automatique lorsque la valeur repasse sous le seuil.

Le module intègre enfin une logique d'optimisation de rendu (mise à jour uniquement quand nécessaire) pour limiter les artefacts visuels et améliorer la stabilité de l'affichage.

3.3.6 Module 6 Logique applicative (orchestration)

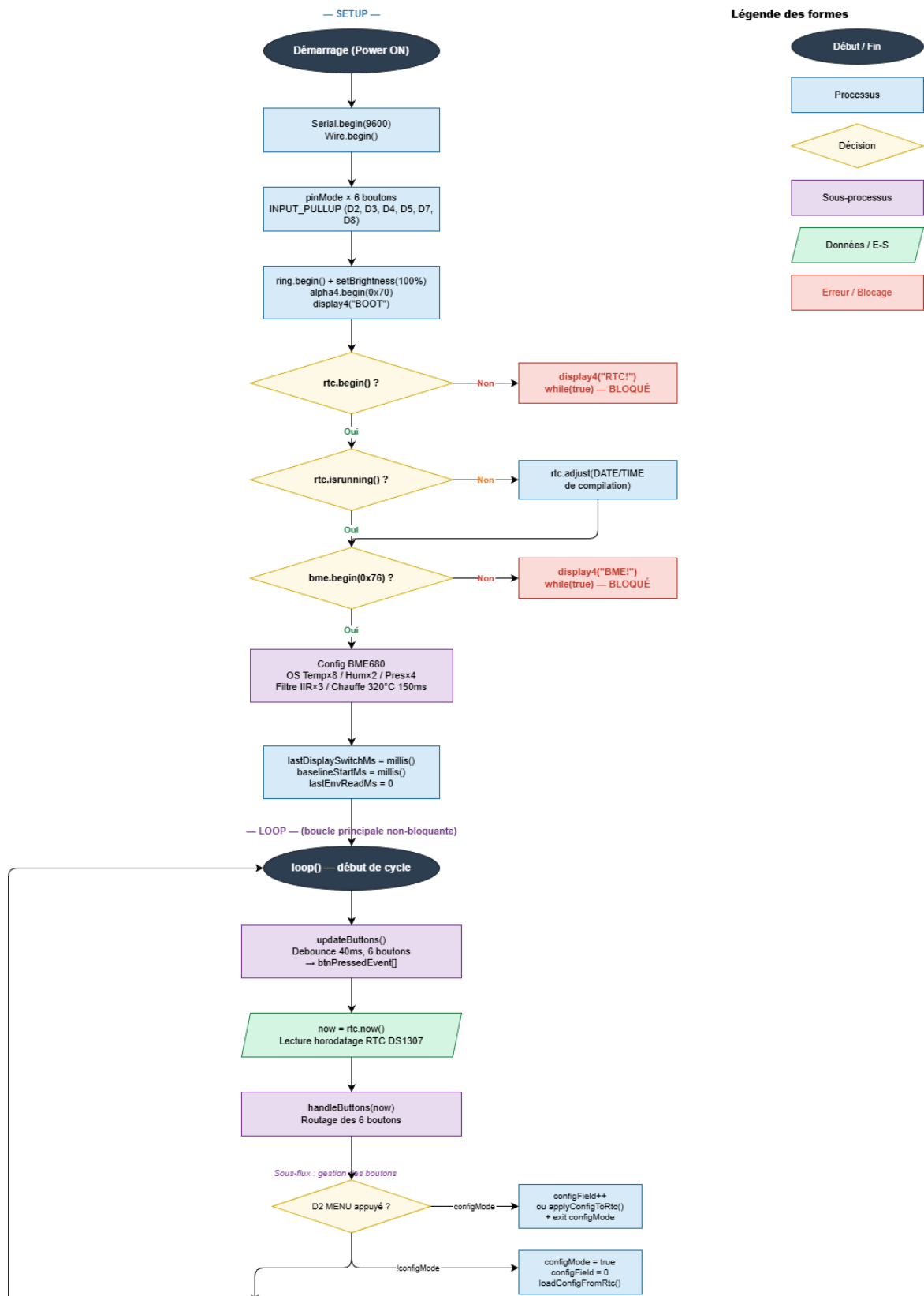
Ce module central, exécuté dans loop(), coordonne l'ensemble :

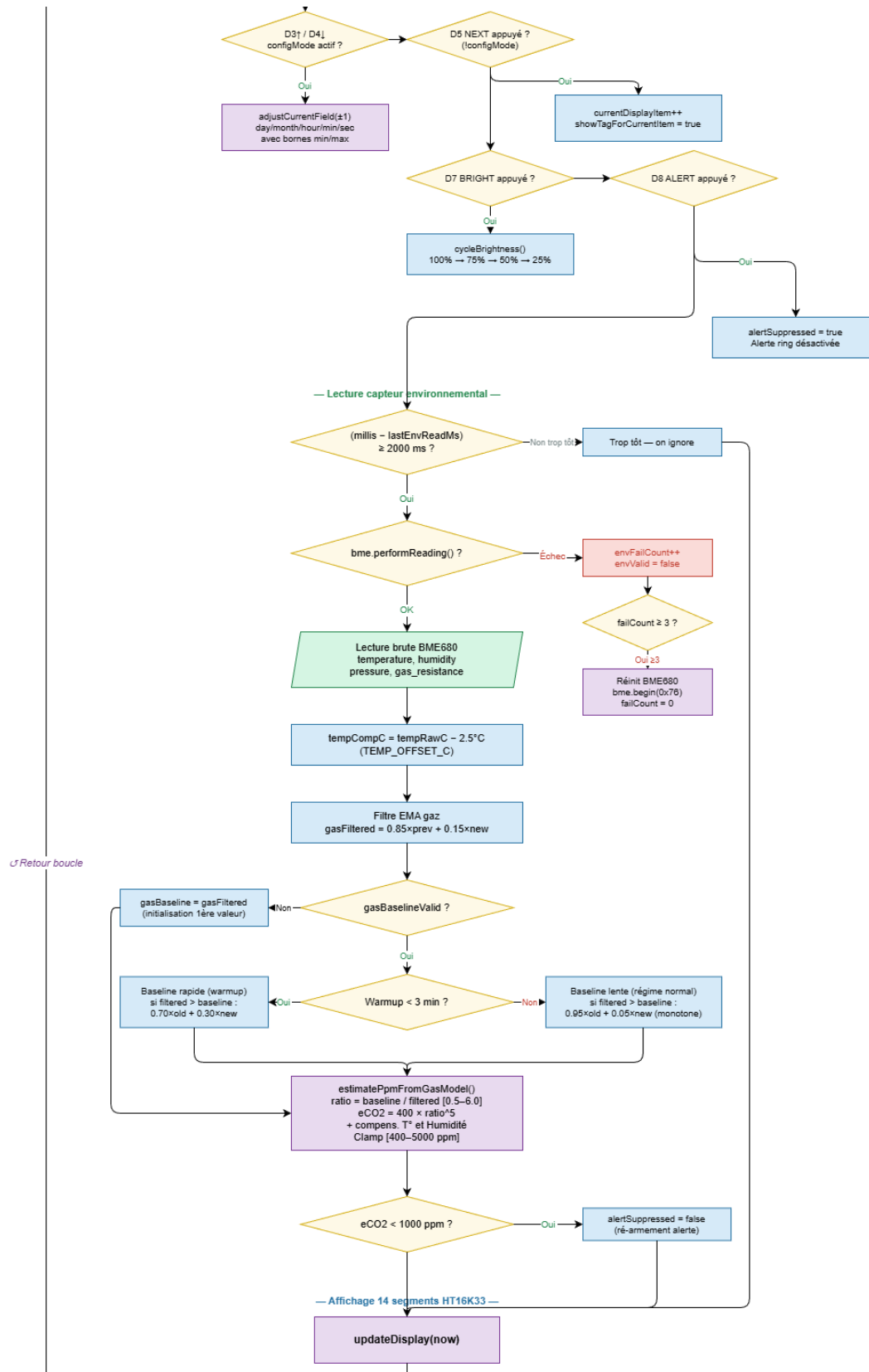
1. lecture des boutons,
2. acquisition capteurs,
3. mise à jour affichage 14 segments,
4. rendu de l'anneau selon l'état courant (normal, configuration, alerte).

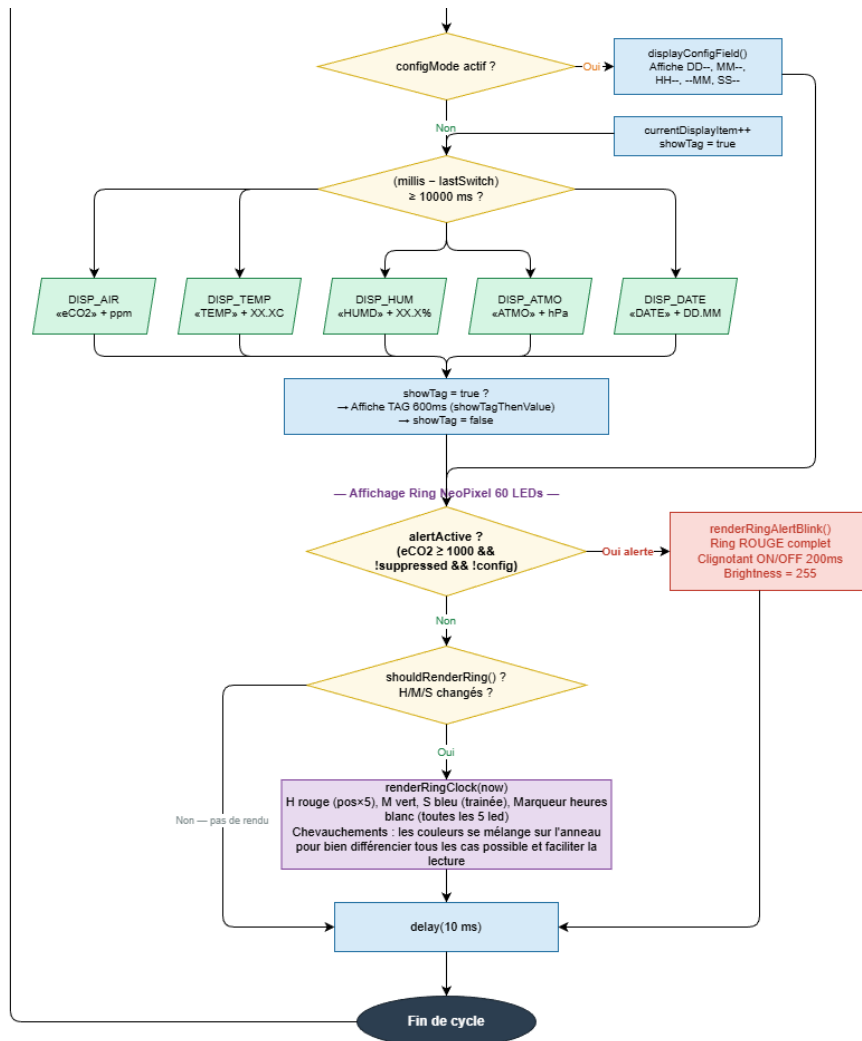
Cette orchestration garantit un fonctionnement réactif, modulaire et maintenable.

3.4 Diagrammes de flux

TPI 2026 — Horloge LED avec capteurs environnementaux
main.cpp — Diagramme de flux complet







3.5 Stratégie de test

Afin de garantir le bon fonctionnement de l'ensemble du système avant la livraison finale, une stratégie de test structurée a été mise en place. Celle-ci suit une progression logique allant du composant individuel vers le système complet, permettant d'isoler et de corriger les problèmes au plus tôt dans le cycle de développement.

3.5.1 Types de tests et ordre d'exécution

Les tests seront réalisés en quatre phases successives, dans un ordre qui va du plus simple au plus complexe.

3.5.1.1 Phase 1 Tests unitaires par composant

Avant toute intégration, chaque composant est testé individuellement à l'aide d'un programme Arduino dédié et minimaliste. L'objectif est de vérifier que le composant répond correctement et que le câblage est valide, sans interférence des autres modules.

L'ordre de test sera le suivant :

- RTC DS1307 : vérification de la lecture et de l'écriture de l'heure, contrôle de la précision après redémarrage avec pile en place.
- BME680 : vérification des quatre mesures (température, humidité, pression, COV), contrôle de la stabilisation des valeurs après le temps de chauffe du capteur.
- HT16K33 : affichage de caractères de test sur les quatre digits, vérification de la lisibilité et de la réactivité de l'écran.
- NeoPixel : allumage séquentiel de chaque LED du cercle (de la LED 0 à la LED 59), vérification des couleurs RGB, test de la luminosité à différents niveaux.
- Boutons poussoirs : test de chaque bouton un par un, vérification de la détection de l'appui et du relâchement, contrôle de l'efficacité du debounce logiciel.

3.5.1.2 Phase 2 Tests d'intégration par fonctionnalité

Une fois les composants validés individuellement, on teste les fonctionnalités complètes du système, en combinant les modules entre eux :

- Affichage de l'heure en temps réel sur l'anneau NeoPixel via le RTC.
- Affichage alterné des données environnementales sur le HT16K33 via le BME680.
- Réglage manuel de l'heure via les boutons dédiés.
- Changement de mode d'affichage et ajustement de la luminosité via les boutons correspondants.
- Déclenchement de l'alerte visuelle LED lorsque le seuil de qualité de l'air est dépassé.

3.5.1.3 Phase 3 Tests d'intégration globale

L'ensemble du système est testé en conditions réelles de fonctionnement, tous les modules actifs simultanément. On vérifie que les différentes fonctionnalités cohabitent sans conflit, que le bus I²C ne génère pas d'erreurs de communication, et que les performances générales restent stables dans le temps.

3.5.1.4 Phase 4 Tests de robustesse et cas limites

On soumet le système à des situations particulières pour vérifier sa solidité :

- Coupure et rétablissement de l'alimentation (vérification que l'heure reste correcte grâce au RTC).
- Simulation d'une mauvaise qualité de l'air (valeur COV dépassant le seuil défini) pour vérifier le déclenchement de l'alerte.
- Maintien d'un bouton appuyé de manière prolongée pour vérifier l'absence de comportement erratique.
- Fonctionnement continu pendant une période prolongée pour détecter d'éventuelles dérives ou fuites mémoire.

3.5.2 Moyens à mettre en œuvre

Les tests seront réalisés avec les moyens suivants :

- CLion avec PlatformIO : pour compiler et téléverser les programmes de test sur la carte Arduino, et visualiser les sorties via le moniteur série.
- Moniteur série (Serial Monitor) : outil principal de diagnostic, il permettra d'afficher en temps réel les valeurs lues par les capteurs, les états des boutons et les éventuels messages d'erreur.
- Multimètre : utilisé lors de la phase de câblage pour vérifier la continuité des connexions et l'absence de court-circuit avant toute mise sous tension.
- Observation visuelle directe : pour valider le comportement de l'anneau LED, la lisibilité de l'affichage alphanumérique et la réactivité des boutons.
- Journal de test : chaque test sera consigné dans le journal de travail avec son résultat (succès/échec), les éventuelles anomalies constatées et les corrections apportées.

3.5.3 Couverture des tests

Les tests ne seront pas exhaustifs au sens strict du terme. En effet, tester l'intégralité des combinaisons possibles d'états (toutes les heures, toutes les valeurs de capteurs, toutes les séquences de boutons) serait irréaliste dans le temps imparti pour ce TPI.

La stratégie adoptée est une couverture fonctionnelle ciblée : on teste les cas nominaux (fonctionnement normal attendu), les cas limites identifiés comme critiques (seuil d'alerte, coupure de courant, debounce) et les cas les plus susceptibles de révéler un défaut d'intégration. Les combinaisons peu probables ou sans impact sur la sécurité du système ne seront pas testées.

Ce choix est justifié par la nature du projet : il s'agit d'un appareil de visualisation et d'alerte, non d'un système critique où une défaillance aurait des conséquences graves. Une couverture fonctionnelle bien ciblée est donc suffisante et proportionnée aux enjeux.

3.5.4 Données de test

Les données de test seront majoritairement des données réelles, collectées directement par les capteurs en conditions d'utilisation normales. Pour les tests unitaires du BME680, les valeurs lues (température, humidité, pression) seront comparées à des références connues (température de la pièce vérifiée avec un thermomètre externe, humidité approximative selon la saison).

Pour le test de l'alerte visuelle, dans la mesure où il n'est pas aisé de dégrader volontairement la qualité de l'air de manière contrôlée, le seuil de déclenchement sera temporairement abaissé dans le code afin de simuler une situation d'alerte sans avoir à créer réellement de mauvaises conditions environnementales.

Pour le RTC, l'heure sera réglée manuellement à une valeur de référence et confrontée à l'heure réelle après plusieurs minutes de fonctionnement afin d'évaluer la dérive.

3.5.5 Testeurs extérieurs

Les tests seront principalement réalisés par le candidat lui-même tout au long de la phase de développement. Aucun testeur extérieur formel n'est prévu dans le cadre de ce TPI.

Cependant, une validation informelle pourra être effectuée par le chef de projet Raphaël Favre lors des points de suivi réguliers, notamment pour valider que le comportement visuel de l'horloge et des alertes correspond aux attentes du cahier des charges. Les experts pourront également observer le fonctionnement du système lors de la présentation finale du 2 ou 3 juin 2026.

3.6 Planification finale

A voir à la fin du projet

3.7 Dossier de conception

Le dossier de conception regroupe l'ensemble des décisions et spécifications techniques prises avant et pendant la phase de réalisation. Il doit être suffisamment détaillé pour permettre à une tierce personne de réaliser le projet sans avoir accès au code source.

3.7.1 Choix du matériel

Les choix matériels ont été analysés en détail dans la section 2.2. Le tableau ci-dessous résume les décisions retenues pour ce projet :

Composant	Référence retenue	Rôle principal	Interface
Microcontrôleur	Arduino UNO REV3 (ATmega328P)	Orchestration de tout le système	—
Affichage horloge	4× Adafruit NeoPixel 1/4 anneau (60 LED WS2812B)	Horloge analogique + alerte visuelle	Signal numérique D6
Capteur environnemental	Adafruit BME680	Température, humidité, pression, COV	I ² C (0x76)
Horloge temps réel	Adafruit RTC DS1307 + pile CR1220	Maintien de l'heure sans alimentation	I ² C (0x68)

Affichage données	Adafruit 4×14 segments HT16K33	Affichage textuel des mesures	I ² C (0x70)
Interface utilisateur	6 boutons poussoirs	Réglage heure, luminosité, mode, alerte	Digital D2/D3/D4/D5/D7/D8
Alimentation / upload	Câble USB-B 2.0 vers USB-A 2.0	Alimentation + téléversement du code	USB
Prototypage	Breadboard + jumpers mâle-mâle	Connexions sans brasure	—
Station de travail	PC CPNV Dell OptiPlex 7000	Développement, rédaction, versioning	—

3.7.2 Choix des outils logiciels

3.7.2.1 Pour la réalisation (développement)

Outil	Version / Référence	Usage
CLion	JetBrains CLion 2026.1	IDE principale de développement C++
PlatformIO	Plugin CLion	Gestion des bibliothèques, compilation, upload
Git	2.51.0.windows.1	Versioning du code source
GitHub	github.com	Dépôt distant, sauvegarde, livraison
Fritzing	0.9.3b	Schéma de câblage breadboard et schématique
Microsoft Word	Office 365	Rédaction du rapport et du journal de travail
Microsoft PowerPoint	Office 365	Présentation orale finale

3.7.2.2 Bibliothèques Arduino (via PlatformIO)

Catégorie	Composant / Bibliothèque	Version	Utilité
Platform	atmelavr	5.1.0	Plateforme PlatformIO pour les microcontrôleurs AVR d'Atmel/Microchip (Arduino Uno, Nano, Mega, etc.).
Framework	framework-arduino-avr	5.2.0	Framework Arduino officiel pour les cartes AVR. Fournit les fonctions setup(), loop(), digitalWrite(), etc.
Outil	tool-avrdude	1.60300.200527	Logiciel utilisé pour téléverser le programme sur la carte Arduino via USB.
Compilateur	toolchain-atmelavr	1.70300.191015	Ensemble des outils de compilation AVR (gcc, g++, linker...). Transforme le code C/C++ en programme exécutable pour le microcontrôleur.

Bibliothèques principales

Bibliothèque	Version	Utilité
Adafruit BME680 Library	2.0.6	Gestion du capteur environnemental BME680 (température, humidité, pression atmosphérique et qualité de l'air).
Adafruit LED Backpack Library	1.5.1	Contrôle des afficheurs et matrices LED utilisant les contrôleurs HT16K33.
Adafruit NeoPixel	1.15.4	Contrôle des LEDs RGB adressables type WS2812 / NeoPixel.

Dépendances des bibliothèques

Bibliothèque	Version	Utilité
Adafruit BusIO	1.17.4	Simplifie la communication I2C et SPI avec les composants Adafruit.

Adafruit GFX Library	1.12.6	Bibliothèque graphique de base pour dessiner du texte, des formes et des images sur écrans OLED/TFT.
Adafruit SSD1306	2.5.16	Gestion des écrans OLED SSD1306 en I2C ou SPI.
Adafruit Unified Sensor	1.1.15	Interface commune pour différents capteurs Adafruit.
Adafruit GPS Library	1.7.5	Communication avec les modules GPS Adafruit.
RTCLib	2.1.4	Gestion des modules horloge temps réel (RTC) comme DS1307 ou DS3231.
WaveHC	1.0.5	Lecture de fichiers audio WAV depuis une carte SD sur Arduino.
Adafruit ILI9341	1.6.3	Contrôle des écrans TFT utilisant le contrôleur ILI9341.
Adafruit STMPE610	1.1.6	Gestion du contrôleur tactile STMPE610.
Adafruit TSC2007	1.1.2	Gestion du contrôleur tactile TSC2007.
Adafruit SH110X	2.1.14	Gestion des écrans OLED basés sur le contrôleur SH110X.
Adafruit TouchScreen	1.1.6	Lecture des écrans tactiles résistifs.

3.7.3 Choix du système d'exploitation

3.7.3.1 Pour la réalisation

Le développement est réalisé sous Windows 11 (PC CPNV). CLion avec PlatformIO est compatible Windows nativement. Les drivers USB pour l'Arduino UNO (CH340 ou ATmega16U2) sont installés automatiquement par Windows Update ou manuellement via le site Arduino.

3.7.3.2 Pour l'utilisation

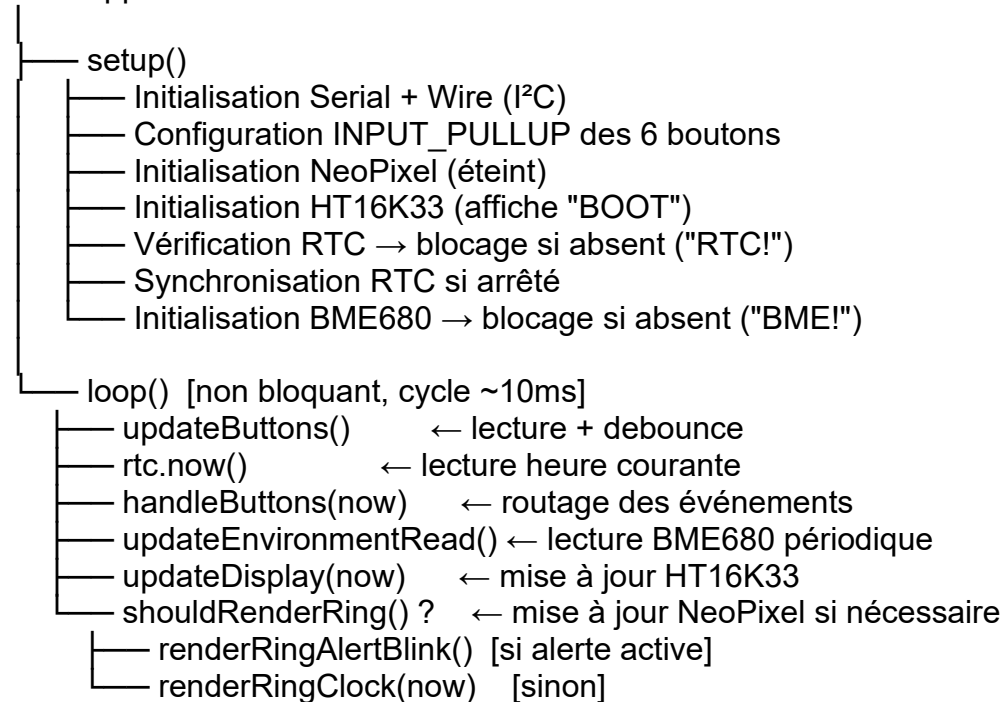
Le système embarqué ne repose sur **aucun système d'exploitation**. Le microcontrôleur ATmega328P exécute le firmware directement sur le hardware (bare-metal), sans couche OS. La boucle principale loop() assure l'ordonnancement de toutes les tâches de manière non-bloquante via millis().

3.7.4 Architecture du programme

Le programme est structuré en un fichier unique main.cpp organisé en fonctions modulaires cohérentes. L'architecture repose sur deux fonctions Arduino standard setup() et loop() complétées par six blocs fonctionnels indépendants.

3.7.4.1 Vue d'ensemble de l'architecture

main.cpp



3.7.5 Découpage modulaire

Le programme est découpé en 6 modules fonctionnels logiques, tous dans main.cpp :

3.7.5.1 Module 1 Gestion des boutons (updateButtons, consumePress)

Entrées : broches D2, D3, D4, D5, D7, D8 (INPUT_PULLUP)

Sorties : tableau btnPressedEvent[6] — un booléen par bouton, mis à true lors d'un front descendant valide

Traitement :

- Lecture brute de chaque broche à chaque cycle
- Comparaison avec l'état précédent
- Si changement détecté → démarrage de la fenêtre anti-rebond (40ms)
- Si état stable depuis 40ms et différent de l'état courant → mise à jour + génération d'événement si LOW

Contrainte : la fonction `consumePress(idx)` consomme l'événement de façon exclusive — un appui ne peut déclencher qu'une seule action.

3.7.5.2 Module 2 Gestion temporelle (`initRTC`, `loadConfigFromRtc`, `applyConfigToRtc`, `displayConfigField`, `adjustCurrentField`, `onMenuButton`)

Entrées : module RTC DS1307 via I²C (adresse 0x68), événements boutons D2/D3/D4

Sorties : objet `DateTime now` utilisé par le reste du programme, écriture RTC via `rtc.adjust()`

Traitement :

- Lecture de l'heure toutes les itérations de `loop()` via `rtc.now()`
- Mode configuration activé par D2 (MENU) :
 - Champs éditables dans l'ordre : jour → mois → heure → minute → seconde
 - D3 (UP) incrémente le champ actif dans ses bornes valides
 - D4 (DOWN) décrémente
 - D2 (MENU) valide et passe au champ suivant au 5e champ : écriture RTC et sortie du mode config
- Gestion des bornes : jours selon mois/année, heures 0-23, minutes/secondes 0-59

3.7.5.3 Module 3 Capteur environnemental (`updateEnvironmentRead`, `estimatePpmFromGasModel`)

Entrées : BME680 via I²C (adresse 0x76), timer interne `lastEnvReadMs`

Sorties : variables globales `tempCompC`, `envHumPct`, `envPressureHpa`, `envPpmEstimate`, `envValid`

Traitement (toutes les 2000ms) :

1. Lecture brute BME680 : température, humidité, pression, résistance gaz (Ohm)
2. Compensation température : $\text{tempCompC} = \text{tempRawC} - 2.5^{\circ}\text{C}$ (offset fixe pour corriger l'auto-échauffement du capteur)
3. Filtrage EMA du signal gaz : $\text{gasFiltered} = 0.85 \times \text{old} + 0.15 \times \text{new}$
4. Baseline adaptative (référence "air propre") :
 - a. Phase warmup (< 3 min) : baseline suit rapidement les meilleures valeurs ($0.70 \times \text{old} + 0.30 \times \text{new}$)
 - b. Régime normal : baseline monotone croissante uniquement ($0.95 \times \text{old} + 0.05 \times \text{new}$)
5. Estimation eCO₂ : $\text{ppm} = 400 \times (\text{baseline} / \text{filtered})^5$ puis compensation T/H, clampé [400–5000 ppm]
6. Réarmement alerte si ppm revient sous 1000

Gestion d'erreur : après 3 lectures consécutives en échec → réinitialisation complète du BME680

3.7.5.4 Module 4 Affichage alphanumérique (updateDisplay, display4, displayTemp, displayHumidity, displayDateDDMM, displayNumber4, showTagThenValue)

Entrées : variables environnementales, DateTime now, configMode, currentDisplayItem

Sorties : affichage HT16K33 via I²C (adresse 0x70)

Traitement mode normal :

- Rotation automatique des pages toutes les 10 secondes dans l'ordre : eCO2 → TEMP → HUMD → ATMO → DATE
- Avant chaque valeur : affichage du TAG pendant 600ms (non bloquant via tagDisplayUntilMs)
- Bouton D5 (NEXT) : avance manuellement à la page suivante avec reset du timer

Traitement mode configuration :

- Affichage du champ en cours d'édition uniquement :
 - Jour : DD--
 - Mois : MM--
 - Heure : HH--
 - Minute : --MM
 - Seconde : SS--

Formats d'affichage par page :

Page	TAG	Format valeur	Exemple
eCO2	eCO2	Entier sur 4 chars	1234
Température	TEMP	XX.XC avec virgule décimale	22.5C
Humidité	HUMD	XX.X% avec virgule décimale	45.2%
Pression	ATMO	Entier hPa sur 4 chars	1013
Date	DATE	DD.MM avec point décimal	08.05

3.7.5.5 Module 5 Anneau NeoPixel (renderRingClock, renderRingAlertBlink, shouldRenderRing, cycleBrightness)

Entrées: DateTime now, envPpmEstimate, alertSuppressed, brightnessStep, configMode

Sorties : 60 LEDs WS2812B via D6

Traitement mode horloge normal :

Le rendu du ring est réalisé uniquement lorsqu'un changement utile est détecté (heure/minute/seconde), afin de limiter les rafraîchissements inutiles et le scintillement.

- Position heure : (heure % 12) × 5 (sur 60 positions)
- Position minute : minute (0–59)
- Secondes : arc cumulatif de LED0 jusqu'à la seconde courante
- Marqueurs horaires : LED de repère sur chaque multiple de 5 (0, 5, 10, ..., 55)

Gestion des superpositions (masque binaire)

Le module utilise un masque 4 bits :

- bit0 = Heure (H)
- bit1 = Minute (M)
- bit2 = Marqueur blanc d'heure (W)
- bit3 = Seconde (S)

Masque	Composantes	Couleur affichée
0001	H	Rouge (100,0,0)
0010	M	Vert (0,100,0)
0100	W	Blanc faible (35,35,35)
1000	S	Bleu (0,0,100)
0011	H+M	Jaune (100,100,0)
0101	H+W	Rouge (100,0,0)
1001	H+S	Magenta (100,0,100)
0110	M+W	Vert clair (40,170,55)
1010	M+S	Cyan-vert (0,110,35)
1100	W+S	Bleu clair (40,60,170)
0111	H+M+W	Jaune (100,100,0)
1011	H+M+S	Blanc (100,100,100)
1101	H+W+S	Magenta (100,0,100)
1110	M+S+W	Cyan-vert clair (35,170,95)
1111	H+M+S+W	Blanc (100,100,100)

Luminosité

cycleBrightness() applique 4 paliers via D7 (BRIGHT) : 100% → 75% → 50% → 25% (puis boucle)

Traitement mode alerte (eCO2 >= 1000 ppm)

- Anneau complet en rouge clignotant ON/OFF toutes les 200 ms

- Luminosité forcée à 255 pendant l'alerte
- D8 (ALERT) désactive l'alerte tant que le seuil n'est pas repassé sous la limite
- Réarmement automatique lorsque $eCO_2 < 1000$ ppm

Optimisation d'affichage

`shouldRenderRing()` évite les rendus inutiles :

- En mode normal : rendu uniquement lors d'un changement temps
- En mode alerte : rendu uniquement au changement de phase de clignotement

3.7.5.6 Module 6 Orchestration (loop, handleButtons)

Entrées : tous les modules ci-dessus

Sorties : coordination du rendu et des états

Ordre d'exécution à chaque cycle (~10ms) :

1. `updateButtons()` → mise à jour des états boutons
2. `now = rtc.now()` → lecture horodatage RTC
3. `handleButtons(now)` → routage des 6 boutons :
 - D2 → `onMenuButton()` : entrée/navigation/sortie config
 - D3 → `adjustCurrentField(+1)` : incrément champ (si `configMode`)
 - D4 → `adjustCurrentField(-1)` : décrément champ (si `configMode`)
 - D5 → changement page HT16K33 (si `!configMode`)
 - D7 → `cycleBrightness()` : cycle luminosité NeoPixel
 - D8 → `alertSuppressed = true` : désactivation alerte
4. `updateEnvironmentRead()` → lecture BME680 si délai écoulé
5. `updateDisplay(now)` → mise à jour HT16K33
6. `shouldRenderRing()` ? → rendu NeoPixel si nécessaire
 - └─ `renderRingAlertBlink()` ou `renderRingClock(now)`
7. `delay(10)` → stabilisation cycle

3.7.6 Entrées / sorties des modules

Module	Entrées	Sorties
Boutons	D2, D3, D4, D5, D7, D8 (physique)	<code>btnPressedEvent[6]</code> (logique)
RTC	I ² C 0x68, événements config	<code>DateTime now</code> , écriture RTC
BME680	I ² C 0x76, <code>millis()</code>	<code>tempCompC</code> , <code>envHumPct</code> , <code>envPressureHpa</code> , <code>envPpmEstimate</code> , <code>envValid</code>
HT16K33	Variables env., <code>now</code> , <code>configMode</code>	Affichage I ² C 0x70

NeoPixel	now, envPpmEstimate, brightnessStep, alertSuppressed	60 LEDs D6
Orchestration	Tous	Coordination états + rendu

3.7.7 Variables globales et états du système

Variable	Type	Rôle
configMode	bool	Indique si le système est en mode réglage
configField	uint8_t (0–4)	Champ en cours d'édition (jour/mois/h/min/sec)
alertSuppressed	bool	Alerte désactivée manuellement par l'utilisateur
brightnessStep	uint8_t (0–3)	Niveau de luminosité actuel (100/75/50/25%)
currentDisplayItem	enum (0–4)	Page affichée sur le HT16K33
envPpmEstimate	int	Valeur eCO2 estimée en ppm
envValid	bool	Indique si la dernière lecture BME680 est valide
gasBaselineOhm	float	Référence "air propre" pour le calcul eCO2
gasFilteredOhm	float	Signal gaz lissé (EMA)

3.7.8 Câblage tableau de connexions complet

Composant	Broche composant	Broche Arduino
Breadboard rail +	—	5V
Breadboard rail -	—	GND
RTC DS1307	VCC	Breadboard 5V
RTC DS1307	GND	Breadboard GND
RTC DS1307	SDA	A4
RTC DS1307	SCL	A5

BME680	5V	Breadboard 5V
BME680	GND	Breadboard GND
BME680	SDA	A4 (bus partagé)
BME680	SCL	A5 (bus partagé)
HT16K33	VCC	Breadboard 5V
HT16K33	GND	Breadboard GND
HT16K33	SDA	A4 (bus partagé)
HT16K33	SCL	A5 (bus partagé)
NeoPixel Q1	PWR	Breadboard 5V
NeoPixel Q1	GND	Breadboard GND
NeoPixel Q1	DIN	D6
NeoPixel Q1→Q2	DOUT→DIN	Chaîne directe
NeoPixel Q2→Q3	DOUT→DIN	Chaîne directe
NeoPixel Q3→Q4	DOUT→DIN	Chaîne directe
NeoPixel Q2/Q3/Q4	PWR	Q1 PWR (chaîne)
NeoPixel Q2/Q3/Q4	GND	Q1 GND (chaîne)
Bouton 1	Signal	D2
Bouton 2	Signal	D3
Bouton 3	Signal	D4
Bouton 4	Signal	D5
Bouton 5	Signal	D7
Bouton 6	Signal	D8
Tous boutons	GND (patte A)	Breadboard GND

Résistances pull-up internes activées via INPUT_PULLUP — aucune résistance physique nécessaire.

4 Réalisation

4.1 Dossier de réalisation

4.1.1 Formule pour le calcul de la valeur du CO2 en ppm

4.1.1.1 Lissage du gaz (gasFilteredOhm)

Formule utilisée à chaque nouvelle mesure :

$$\text{gasFilteredOhm} = \text{gasFilteredOhm} * 0.85 + \text{envGasOhm} * 0.15$$

- envGasOhm = mesure brute actuelle du capteur gaz (ex : 615759)
- gasFilteredOhm = ancienne valeur lissée (ex : 613865)
- Nouvelle gasFilteredOhm $\approx 613865 * 0.85 + 615759 * 0.15 = 614149$

4.1.1.2 Baseline (gasBaselineOhm)

Au début :

- gasBaselineOhm = gasFilteredOhm

Ensuite :

- Si gasFilteredOhm > gasBaselineOhm pendant warmup:

$$\text{gasBaselineOhm} = \text{gasBaselineOhm} * 0.70 + \text{gasFilteredOhm} * 0.30$$

- Si gasFilteredOhm > gasBaselineOhm après warmup:

$$\text{gasBaselineOhm} = \text{gasBaselineOhm} * 0.95 + \text{gasFilteredOhm} * 0.05$$

- Sinon, gasBaselineOhm ne change pas.

4.1.1.3 Ratio

$$\text{Ratio} = \text{gasBaselineOhm} / \text{gasFilteredOhm}$$

Exemple avec tes valeurs :

- gasBaselineOhm = 676775
- gasFilteredOhm = 614149

$$\text{Ratio} = 676775 / 614149 = 1.10197$$

Puis on limite :

- Si Ratio < 0.5 => ratio = 0.5
- Si Ratio > 6.0 => ratio = 6.0

4.1.1.4 Calcul PPM principal

$$\text{Ppm} = 400 * \text{pow}(\text{ratio}, 5.0)$$

Avec ratio = 1.10197:

$$\text{Ppm} = 400 * 1.10197^{5.0} \approx 650$$

4.1.1.5 Correction humidité/température

$$\text{Ppm} = \text{ppm} + (\text{envHumPct} - 40.0) * 1.6$$

$$\text{Ppm} = \text{ppm} + (\text{tempCompC} - 22.0) * 3.0$$

Avec :

- envHumPct = 33.5 => (33.5-40)*1.6 = -10.4
- tempCompC = 22.9 => (22.9-22)*3.0 = +2.7

Donc :

$$\text{Ppm} \approx 650 - 10.4 + 2.7 = 642.3$$

4.1.1.6 Limite finale

- Si ppm < 400 => ppm = 400
- Si ppm > 5000 => ppm = 5000

Ici : Ppm~642.

4.2 Erreurs restantes

4.3 Liste des documents fournis

5 Tests

6 Conclusions

7 Annexes

7.1 Résumé du rapport du TPI / version succincte de la documentation

7.2 Sources – Bibliographie

7.3 Journal de travail

7.4 Manuel d'Installation

7.5 Manuel d'Utilisation

7.6 Archives du projet